



Protein sequence-structure tokeniser for diffusion-based neural network

Shahin Zibae¹

Machine Learning MSc

Professor David Jones

Submission date: 31st January 2025

¹**Disclaimer:** This report is submitted as part requirement for the Machine Learning MSc at UCL. It is substantially the result of my own work except where explicitly indicated in the text. The report will be distributed to the internal and external examiners, but thereafter may not be copied or distributed except with permission from the author.

Abstract

Scientific contributions by thousands of computational and experimental scientists over decades of intense efforts have brought protein structure prediction to a seminal stage. Most recently, the field of computational and structural biology received a number of unexpected boosts in the form of novel contributions by the largest technology firms in the world. These companies allocated significant data processing infrastructure and software engineering personnel to research and development in applied structural biology. From around 2019-2020, this combined with particular applications of deep attention-based models and, most recently, probabilistic generative models such as diffusion models.

While the rate of improvements in accuracy has been blisteringly fast in the last 5 years in particular, these have not yet translated directly to a similar level of mechanistic understanding at the molecular level. A tantalising opportunity now presents itself to structural biologists to unravel the deep learning models and exploit their predictive powers in order to thoroughly understand the molecular determinants of protein folding in its entirety. It is likely that this would be greatly aided by the provision of more computationally-efficient deep learning models.

The product of this MSc project can make a very modest, but sound, contribution in this direction, serving primarily as a starting base. Specifically, the goal has been to modify a deep neural network for RNA structure prediction, designed and built in-house, to train on protein sequences instead. This network model uses a protein language model (pLM) embedding for transfer learning to a diffusion model. As a result, it can now be used for establishing benchmarks and further interrogation of the model architecture by ablation studies, trunk modifications, as well as heuristic manipulations to create a streamlined tool that aids systematic research into the molecular determinants of protein folding *in silico*.

Acknowledgements

I am very grateful to Professor David Jones for the opportunity to undertake research in arguably the most fascinating topic at the intersection of cutting-edge deep learning and structural biology.

I thank Dr Shaun Kandathil for his help with deep learning, HPC, CATH, hardware, guidance and advice.

I thank Dr Tiejun Wei for his help with protein language model embeddings and help with HPC amongst other technical topics.

I am grateful to all of the aforementioned for their patience and kind support in general.

Contents

1	Introduction	6
2	Related Work	23
3	Methods	41
4	Experiments	59
5	Results	61
6	Discussion	64

List of Figures

1.1	Schematic of one protein folding from an unfolded non-specific shape to a specific folded form. Backbone atoms depicted only, for simplicity. (Creative commons).	7
1.2	Polypeptide chain represented as a chain of beads. Each bead represents one amino acid. (www.genome.gov).	8
1.3	Atoms of polypeptide forming two planes, with phi and psi torsional angles labelled. R is the amino acid side chain group. (Creative commons).	9
1.4	Chemical and molecular structure of an amino acid (top) and of 4 amino acids polymerised in a polypeptide chain (middle); relation to beads on chain representation of a polypeptide (bottom) (Creative commons).	10
1.5	Folding of a single protein chain. Schematic illustrates how different folding pathways might manifest in terms of protein structure. (Creative commons).	11
1.6	Example of a multiple sequence alignment. A single protein family is alignment according to sequence similarity. Each row is one protein from a different species). (Creative commons).	12
1.7	Early application of shallow neural network for protein secondary structure prediction (Qian & Sejnowski 1988).	13
1.8	Application of CNNs in deep residual networks for protein contact map prediction (Wang et al. 2016).	14
1.9	CNNs in three separate 18-residual block networks, each predicting a separate structure parameter (Greener, Kandathil & Jones 2019).	15
1.10	AlphaFold pipeline. Its deep neural network has 220 residual blocks (composed of CNNs) (Senior et al. 2020).	16
1.11	Scaled dot-product attention (left) run in parallel (right) (Vaswani et al. 2017).	18
1.12	The transformer has 6 identical layers of its encoder (left) and 6 identical layers of its decoder (right) (Vaswani et al. 2017).	18
1.13	Four different generative deep learning models, from top to bottom: generative adversarial networks (GANs), variational autoencoders (VAEs), normalising flows and diffusion models (creative commons).	19
2.1	Root mean squared deviation (discarding 5% outliers) (r.m.s.d ₉₅) in angstroms, shows significantly more accurate structure predictions for AlphaFold than 15 other CASP14 entries (2020) (Figurnov 2021).	25
2.2	Global distance test (GDT) metric showing the relatively large improvement gained by AlphaFold (Figurnov 2021).	25
2.3	Evoformer trunk of AlphaFold 2 (Jumper et al. 2021).	27
2.4	AlphaFold 2 (Jumper et al. 2021).	28

2.5	Generation of embedded representation of by self-supervised machine learning method (left) (Yang et al. 2018) based on doc2vec. Right panel represents transfer learning in which the embedding is used for downstream supervised learning tasks.	29
2.6	Evolutionary scale modelling (ESM) protein language model (Rao et al. 2020).	31
2.7	ESMFold structure prediction using version 2 of ESM protein language model. The folding trunk/block is shown in greater detail on the right (Lin et al. 2022).	31
2.8	Version 3 of ESM protein language model (Hayes et al. 2024).	32
2.9	Schematic of the ProtTrans protein language model pre-training and downstream transfer learning (to protein-level or token-level classification) (Elnaggar et al. 2020).	33
2.10	Schematic of Ankh protein language models and the downstream supervised transfer learning prediction tasks (residue-level or protein-level) (Elnaggar et al. 2023).	34
2.11	Schematic of protein structure diffusion model (Watson et al. 2023).	35
2.12	Blocks of invariant point attention (IPA) learn to denoise the diffuse structure (top) with inputs of constraints: a secondary structure-informed heuristic (top-left) and C_α distance matrix embedded via triangular self-attention before processing (Anand & Achim 2022).	36
2.13	The complete architecture of protein design diffusion model. Noising is performed by the encoder, denoising by the decoder (Lin & AlQuraishi 2023).	37
2.14	Frame representation of a backbone residue (A, left). Denoising process (B, right) (Yim et al. 2023).	38
2.15	Schematic representing overall approach of noising and denoising protein backbone coordinates, followed by a design GNN that generates an amino acid sequence and side-chain coordinates for the generated backbone (Ingraham et al. 2023)	38
2.16	Representation of side-chain ‘superposition’ strategy to enable reasoning about side-chains at same time as backbone atoms during the diffusion process. Collapse to single residue occurring during sample generation, or updated for a given amino acid (Chu et al. 2024).	39
2.17	AlphaFold 3 (Abramson et al. 2024).	40
3.1	Directory structure showing only the most important folders, scripts and data files.	43
3.2	Example of <code>_pdbx_poly_seq_scheme</code> showing all of 12 attribute headers and just the first 3 rows of the values for those attributes.	47
3.3	Example of <code>_atom_site</code> showing all of 20 attribute headers and just the first 3 rows of the values for those attributes.	48
5.1	Train (blue) and validation (orange) loss curves over 2000 epochs.	62
5.2	The first 100 epochs only from Figure 5.1.	62

List of Tables

3.1	Package versions on the two machines used for development and training.	42
3.2	CUDA package versions, builds, and channels (on Rocky Linux machine).	42
3.3	Example of the 8 attributes in 8 columns of a dataframe, as returned by <code>cif_parser.parse_cif()</code> , showing first 13 rows. Column names prefixed with ‘A_’ are from <code>_atom_site</code> . Column names prefixed with ‘S_’ prefix are from <code>_pdbx_poly_seq_scheme</code>	49
3.4	Description of the 8 mmCIF attributes that are parsed and will be used for tokenisation in preparation for model training. Attribute names shown here are the same as in the original mmCIF but pre-fixed with ‘S_’ or ‘A_’ to signify which of the two categories they come from, namely <code>_pdbx_poly_seq_scheme</code> or <code>_atom_site</code> , respectively. These prefixed names are the column names of the dataframes and <code>ssv</code> files.	50
3.5	Example output of <code>parse_tokenise.write_cifs_to_flatfile()</code> , written to <code>ssv</code> , showing first 13 rows.	51
3.6	Descriptions of the 5 tokenised mmCIF values: <code>atomcodes</code> , <code>aaindices</code> , <code>bbindices</code> and <code>target_coords</code> and their expected *dimensions and values for one example PDBid.chain which has 150 residues and 1350 atoms.	53
3.7	Example of what the first 8 values of <code>aacodes</code> , <code>atomcodes</code> , <code>aaindices</code> , <code>bbindices</code> and <code>target_coordinates</code> might look like. (<code>target_coords</code> shown to 3 s.f.)	54
5.1	Train and validation losses of the first 21 epochs, highlighting substantial drops immediately after the first few epochs, which are not overtly obvious from the plots alone.	63

Layout of background

The background is broken up over Introduction and Related Work as follows:

- Introduction:
 - Proteins, protein folding and structure.
 - Protein structure prediction and early neural networks.
 - The transformer, generative models and diffusion models.
- Related Work:
 - Attention models for protein structure.
 - Attention models, MSA and deep learning for protein structure (AlphaFold 2).
 - Embeddings and protein language models.
 - Diffusion models for protein structure.

Chapter 1

Introduction

How proteins fold - A difficult problem, worth solving

Proteins perform a multitude of essential functions inside and/or outside cells in all living things, from single-celled organisms to mammals. The mechanisms of these functions for most, if not all, proteins is closely associated with the structures they fold into *in vivo*.

Protein folding - an autonomous, rapid and precise event

All proteins are linear, unbranched polymers of amino acids, with most folding soon after creation into compact three-dimensional forms. In principle, a protein can rely entirely, with exceptions, on its own amino acid sequence to fold and refold into exactly the same specific folded structure on sub-millisecond timescales (Anfinsen & Haber 1961).

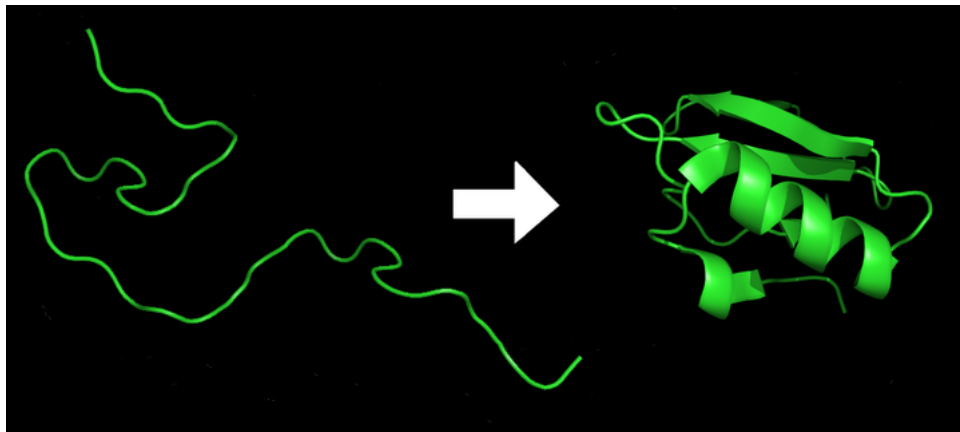


Figure 1.1: Schematic of one protein folding from an unfolded non-specific shape to a specific folded form. Backbone atoms depicted only, for simplicity. (Creative commons).

Hence there is a direct relationship between the amino acid sequence and its functional folded form. Understanding this better can inform how proteins perform their roles in living systems, how mutations that alter the sequence might cause disease, and how to design a wide range of new synthetic peptides with properties beneficial to biomedicine, ecology and material sciences.

Protein sequence and solved structure databanks - decades of painstaking efforts

The task of predicting the three-dimensional structure from its primary structure *in silico* depends on experimentally-determining that amino acid sequence in the first place. Equally important to the development of computational methods is to be able to compare the prediction to the experimentally-solved three-dimensional structure of the protein. The first amino acid sequence was only determined in 1955 (Ryle et al. 1955). The first protein structures were experimentally solved from 1958 (Kendrew et al. 1958; Perutz et al. 1960). Solving the so-called protein-folding problem has been a long-standing goal ever since then and it has seen steady progress over the decades. Specifically, the 'problem' to solve is to predict the atomic structure of a protein, given only its amino acids sequence. Advances in this field have been greatly aided by a benchmarked biennially-run competition, since 1994 - the Critical Assessment for Structure Prediction (Moult et al. 1995). Sequencing of genomes and the resulting proteomes has undergone a dramatic acceleration in the last decade, leading to a massive increase in the number of amino acid sequences available in freely-available protein sequence data banks, first established in 1997 (Bairoch & Apweiler 1997; Bateman et al. 2024). A freely-available protein structure data bank ("Crystallography: Protein Data Bank." 1971; Berman, Henrick & Nakamura 2003) has also continued to accelerate in quantity and quality, though not at the same rate as sequence data banks.

Protein - a polypeptide of amino acids

Every protein is a polypeptide chain constructed of an unbranched sequence of left-handed α -amino acids. There are up to 20 different amino acids that can be found in proteins.

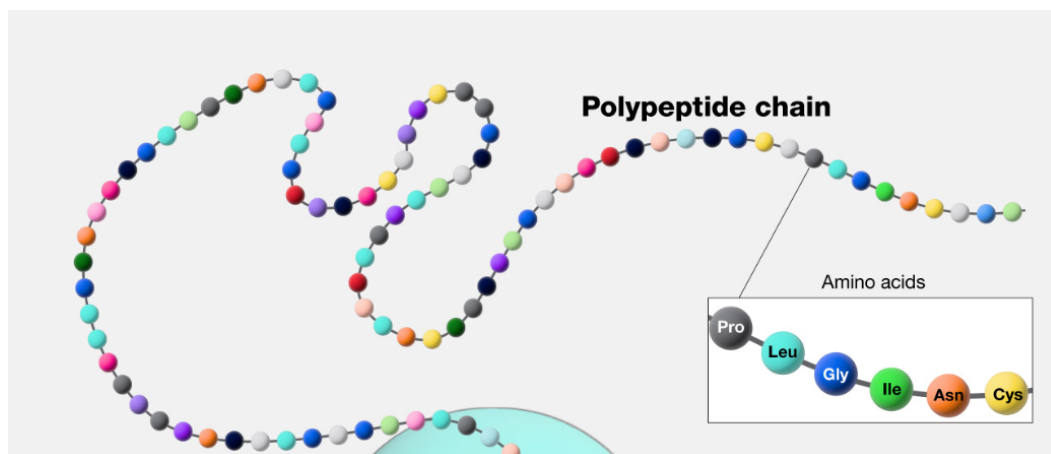


Figure 1.2: Polypeptide chain represented as a chain of beads. Each bead represents one amino acid. (www.genome.gov).

Atomic structure of polypeptides

The polypeptide chain has a repeating sequence of singly-bonded N-C-C backbone, which can therefore rotate freely. However, the C-N (peptide bond) between each N-C-C behaves like a double bond, restricting the rotation at this bond. There is also steric hindrance between the carbonyl oxygen, the hydrogen of the NH group, the hydrogen on the $C\alpha$ carbon and the side chain atoms.

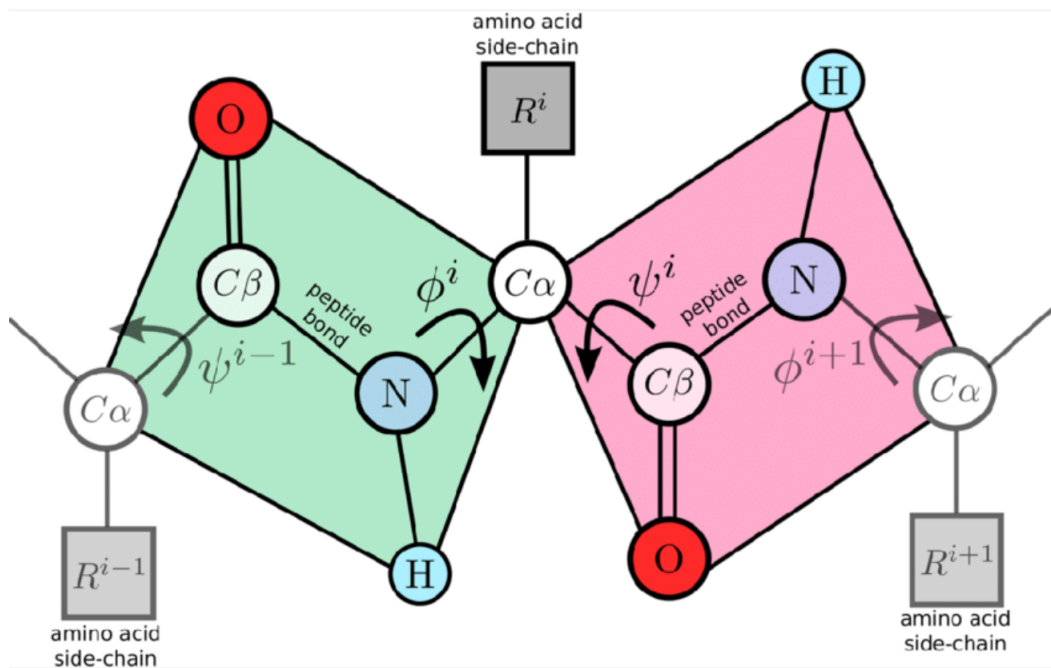


Figure 1.3: Atoms of polypeptide forming two planes, with phi and psi torsional angles labelled. R is the amino acid side chain group. (Creative commons).

Chemical and molecular structure of polypeptides

The representation of proteins as beads-on-a-chain is illustrated here alongside the chemical and molecular structure of amino acids and polypeptides. This helps to bridge the gap between the representation of a protein in Figure 1.2 and its atomic structure as shown in Figure 1.4.

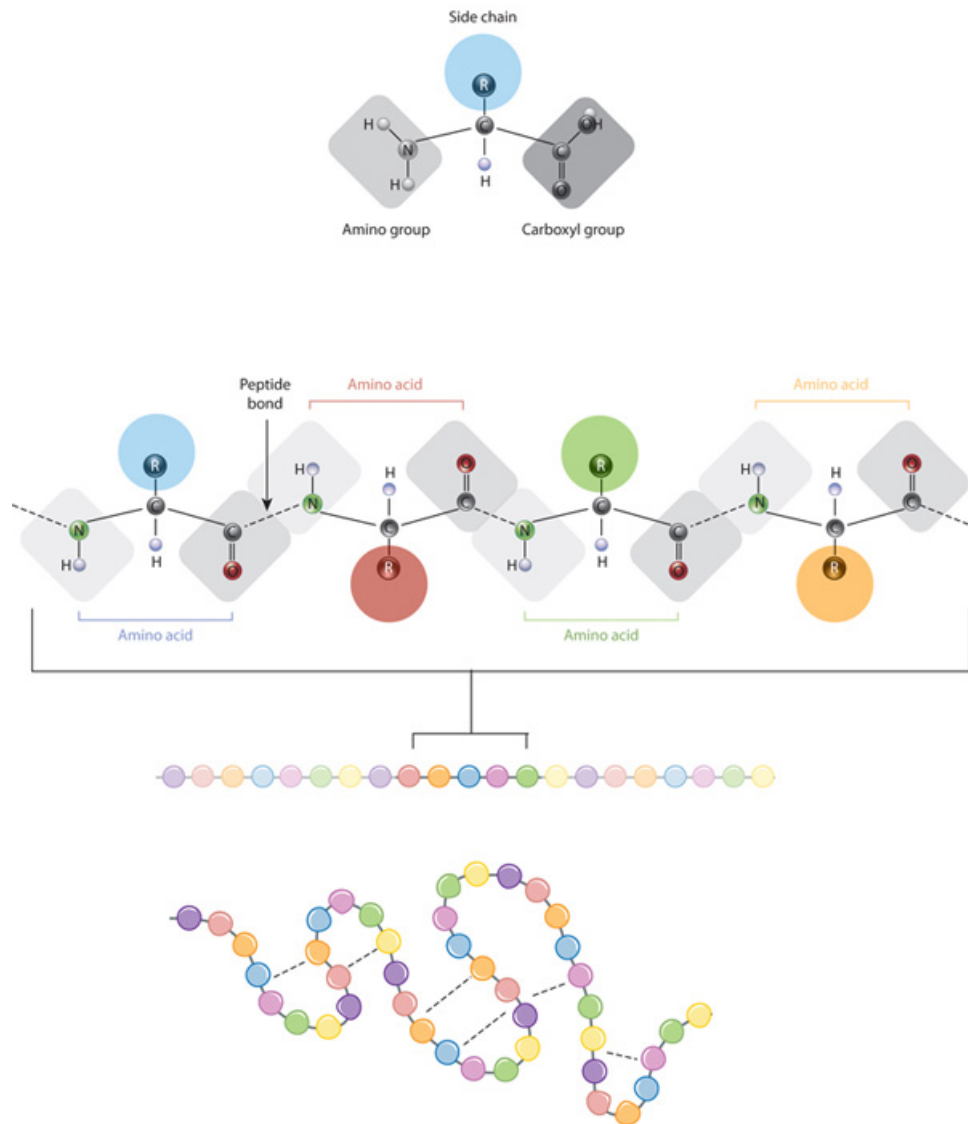


Figure 1.4: Chemical and molecular structure of an amino acid (top) and of 4 amino acids polymerised in a polypeptide chain (middle); relation to beads on chain representation of a polypeptide (bottom) (Creative commons).

Levinthal's paradox and protein folding pathways

Despite these restrictions, the potential for variation in conformations that can be achieved is very large: 3^N for a protein of N amino acids. In theory, it would require a polypeptide chain, of average length, longer than the age of the Universe to sample its entire conformational space, in the search for its final folded fold. It is striking therefore that most proteins fold in seconds or less. This observation is known as Levinthal's Paradox (Levinthal 1969). This has been partially explained as the result of a sequential restriction in the search space as protein traverses a pathway down its folding energy landscape, visually represented by a funnel (Dill & Chan 1997).

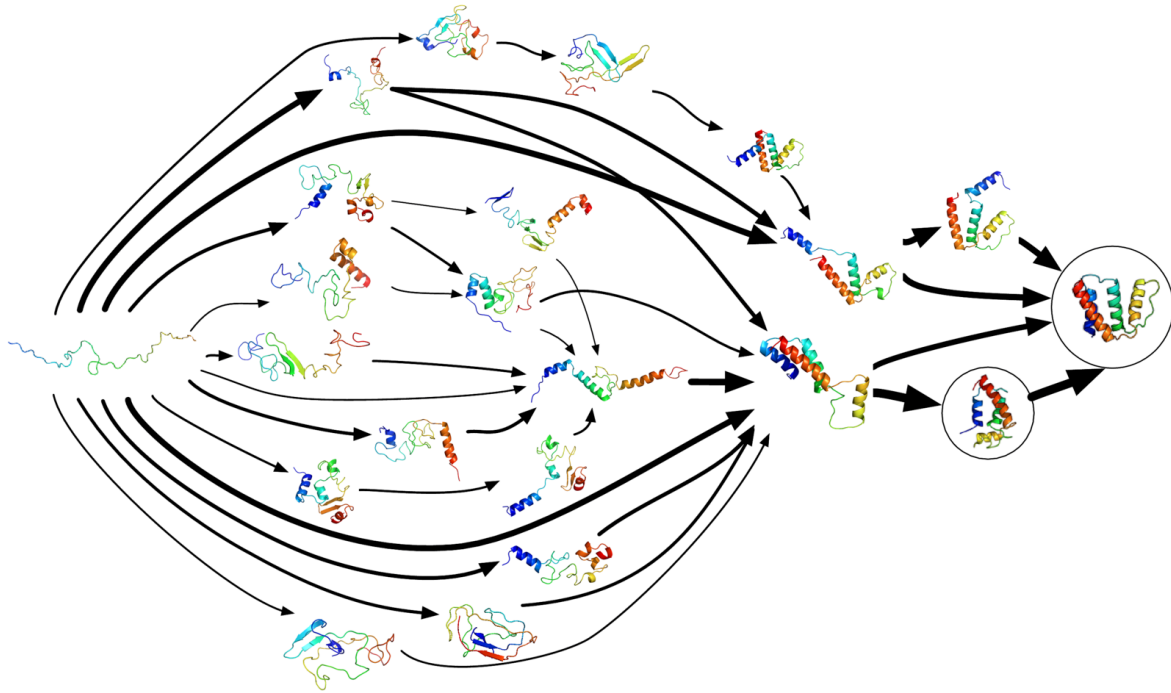


Figure 1.5: Folding of a single protein chain. Schematic illustrates how different folding pathways might manifest in terms of protein structure. (Creative commons).

Protein structure prediction - rational computational techniques

The brightest minds in structural and computational biology have been steadily chipping away at this problem for the last five decades, aided by increases in compute over time. Some notable advances have been due to the development of methods for aligning amino acid sequences of protein families, also known as multiple sequence alignment (MSA) (DF & RF 1987; Gotoh 1999; Katoh et al. 2002). MSA informed the importance of long-range protein contacts and the application of this rationale for prediction further improved results (Thomas, Casari & Sander 1996; Jones et al. 2012).

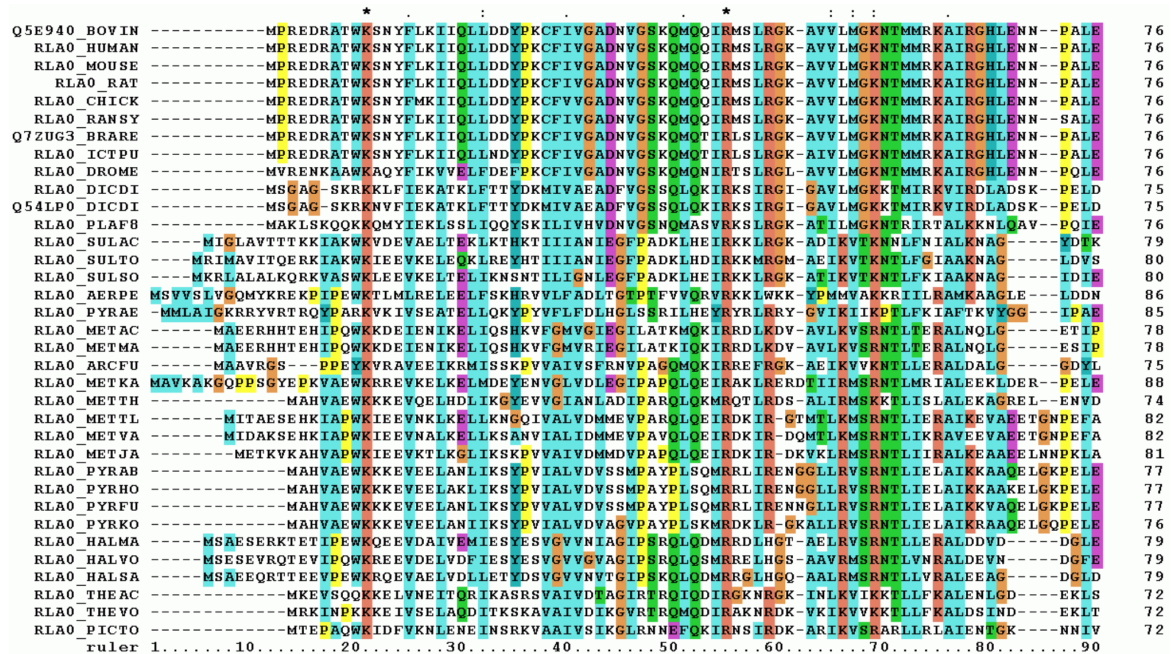


Figure 1.6: Example of a multiple sequence alignment. A single protein family is alignment according to sequence similarity. Each row is one protein from a different species). (Creative commons).

Alignment of proteins is closely align with another greatly impactful method in which protein structure is predicted on the basis of the experimentally-solved structure of another protein with a sufficient number of identical amino acids at the same positions, sometimes referred to as 'homology modelling'. (Blundell, Sibanda & Pearl 1983; Taylor 1986; Blundell et al. 1988; Jones, Taylor & Thornton 1992).

Neural networks in protein structure prediction - early attempts

Shallow neural networks have featured in published protein structure prediction since 1988 (and possibly earlier), applied to secondary structure prediction, as well as the construction of contact maps (Qian & Sejnowski 1988; Bohr et al. 1988; Fredholm et al. 1990; Fariselli & Casadio 1999; Jones 1999; Fariselli et al. 2001).

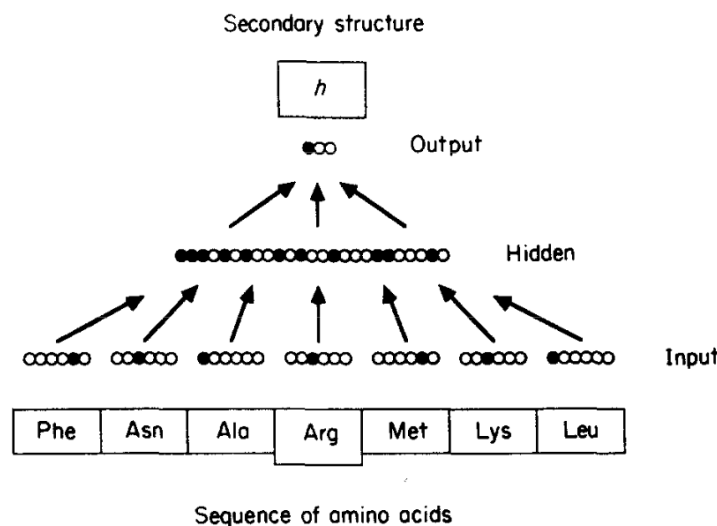


Figure 1.7: Early application of shallow neural network for protein secondary structure prediction (Qian & Sejnowski 1988).

They ranged from small fully-connected neural networks with only a single hidden layer, to an early form of 'recurrent neural networks' (RNNs). Contact predictions were attempted using a 5-layer network of random forest classifiers (Skwark et al. 2014). RNNs designed to be more effective with longer sequences called 'long short-term memory' (LSTMs) (Hochreiter & Schmidhuber 1997; Sutskever, Vinyals & Le 2014) were applied in an end-to-end model for learning protein structural parameters (AlQuraishi 2018).

All the while, computer hardware and its availability to research labs has continued to advance and increase, faster than anticipated. At the same time, the ratio of experimentally-solved protein sequences to their experimentally-solved structures has also continued to increase greatly. The stage was being set for the arrival of an era of effective brute-force applications via developments in computation graphs that combine hierarchical abstractions with non-linearities for function approximation and discriminative tasks, made computationally-feasible by gradient descent, the chain rule and techniques to reduce gradient instability.

Deep learning and protein structure prediction

Unrelated to protein science, in the field of computer vision, sufficiently large computational resources, with sufficiently large, labelled datasets combined with the back-propagation algorithm (Rumelhart, Hinton & Williams 1986) allowed sufficiently deep neural networks to surpass human-level accuracy in image classification tasks (Krizhevsky, Sutskever & Hinton 2012; He et al. 2015). While the inclusion of convolutional neural networks (CNNs) was a crucial component of the neural network architecture (LeCun et al. 1989), it was the introduction of residual connections that facilitated the effective use of deeper networks. CNNs can learn local relative geometries in 2 dimensions by the application of the sliding dot-product operation with a 2D filter, resulting in translationally-equivariant feature extraction. Weights are shared, which has the additional benefit of making CNNs less computationally-demanding than other networks including fully-connected networks. CNNs in 2 residual blocks were used, comparing a range of deep networks from 9 to 60 layers deep, for predicting long-range protein contacts. The accuracy was found to positively correlate with network size (Wang et al. 2016).

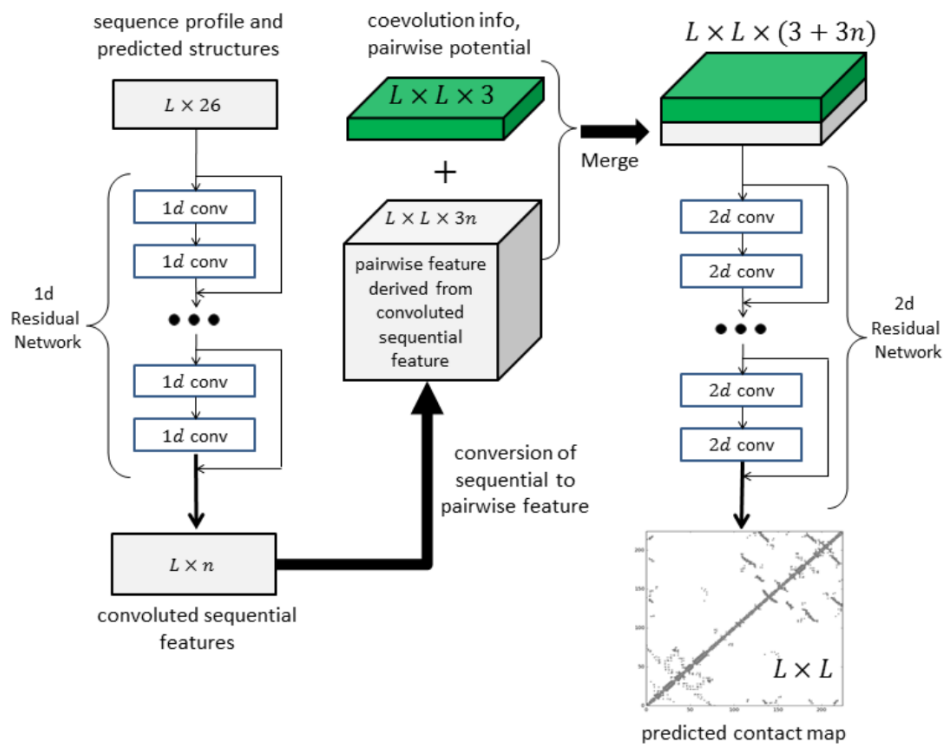


Figure 1.8: Application of CNNs in deep residual networks for protein contact map prediction (Wang et al. 2016).

Deep learning and protein structure prediction

Each of three models of 18 residual blocks, also constructed of CNNs, were used to predict contact distances, H-bond formations and torsional angles (Kandathil, Greener & Jones 2019; Greener, Kandathil & Jones 2019).

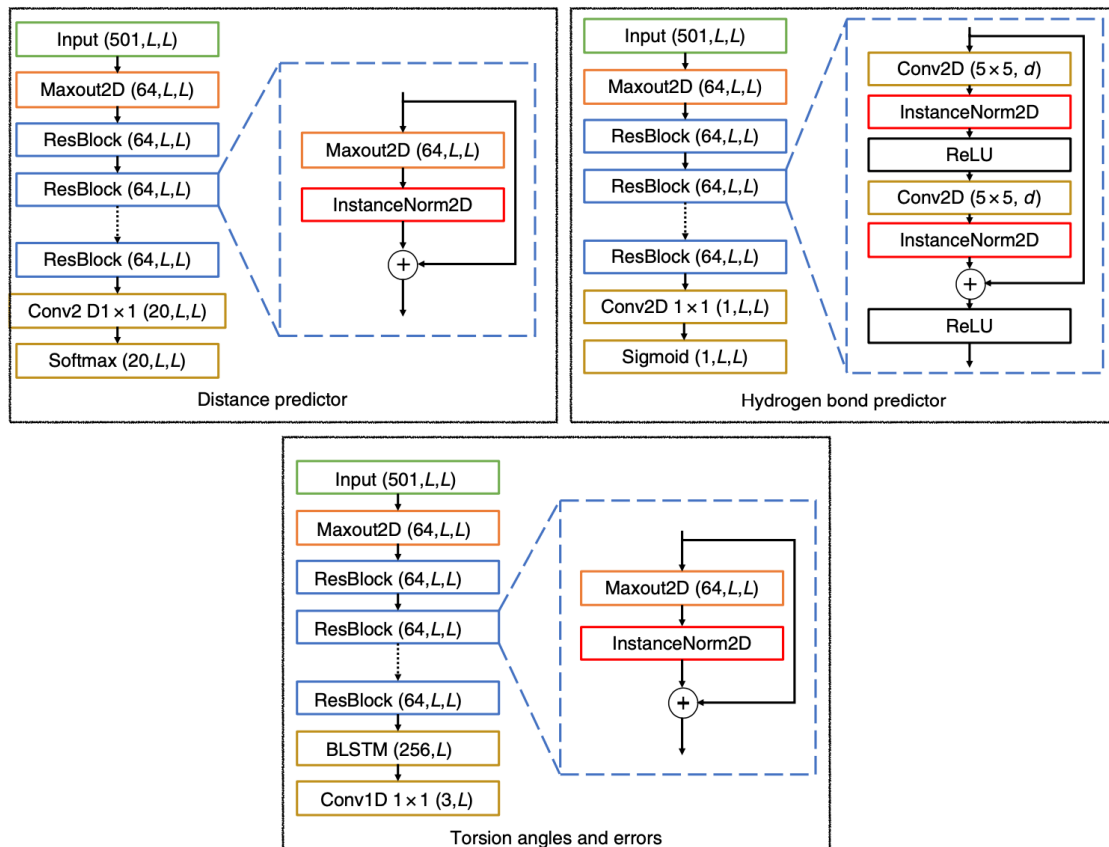


Figure 1.9: CNNs in three separate 18-residual block networks, each predicting a separate structure parameter (Greener, Kandathil & Jones 2019).

Deep learning and protein structure prediction

The first official applications of deep learning began to appear at the 13th CASP protein structure competition, primarily featuring LSTM and CNN trunks (Kryshtafovych et al. 2019). This included the first foray into the field of protein structure prediction by one of the largest technology companies in the world (Senior et al. 2020). The protein structure prediction pipeline consisted of generating MSAs that are fed into a deep network of 220 residual blocks also containing CNNs (Figure 1.10), and trained on the objective to minimise the mean force potential of the stochastically-sampled structures for each protein sequence.

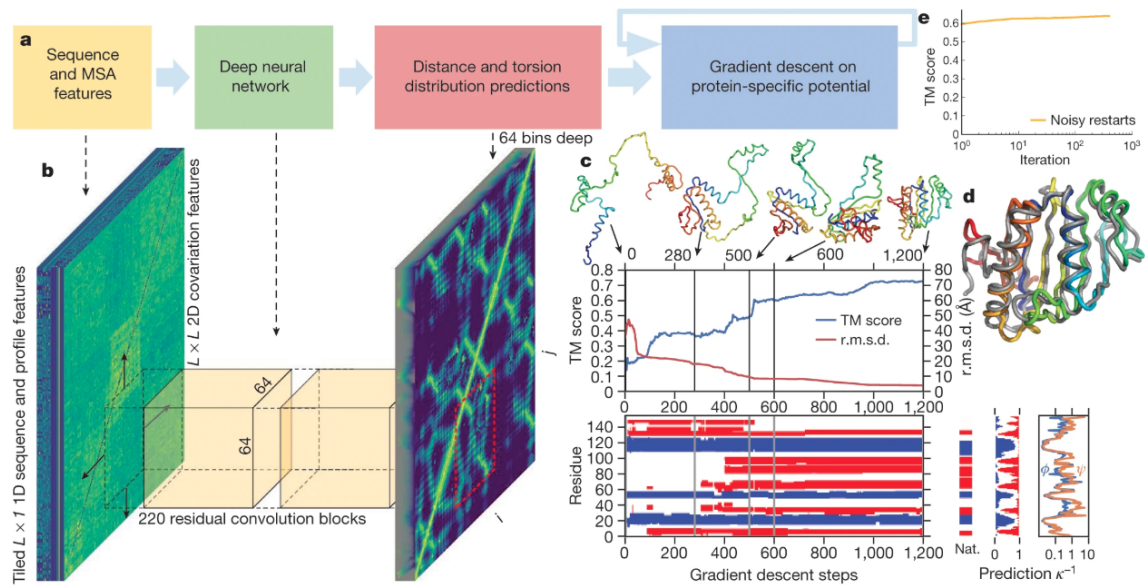


Figure 1.10: AlphaFold pipeline. Its deep neural network has 220 residual blocks (composed of CNNs) (Senior et al. 2020).

Attention models

(Attention models are discussed here in Introduction. Their relevance to protein structure prediction will be discussed in Related Work as well). Attention models were developed to address the limitations of RNNs for longer windows of words and fixed-length vectors, thereby improving contextual understanding in natural language (Bahdanau, Cho & Bengio 2014). The transformer was born out of the demonstration that attention alone was sufficient and superior to RNNs (Vaswani et al. 2017). An encoder-decoder architecture takes ‘vanilla’ word embeddings and trains with these to convert them into contextualised embeddings with ‘deep grammar’ understanding in the encoder part. The decoder part performs self-supervised learning by predicting the next word based on the preceding words. The introduction of the transformer and related attention models in many different deep learning applications, often replacing RNNs and CNNs, has correlated with improved results. Protein structure prediction is no exception to this. The impact of attention models warrants including more detail (from the original transformer paper). The transformer computes a simple similarity metric between two embedding vectors: the ‘query’ embedding which represents what part(s) of the input to focus on, the ‘key’ embedding which represents identifiers for the parts of the input. The similarity metric is computed by a scaled-dot product (Figure 1.11). The result is converted to probabilities by a softmax function (equation 1.1). These are called the ‘attention weights’. A second dot product is performed between the attention weights and a third embedding vector, ‘values’ which are the actual input values. This completes the attention operation producing a weighted sum of relevant values.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (1.1)$$

Attention function: where Q , K and V are the query, key and value embedding vectors (typically 3-dimensional vectors), d_k is the dimension of the K vector.

The transformer utilises ‘self-attention’, a mechanism whereby each part of a single sequence attends to every other part of the same sequence, resulting in a representation of that sequence. The three embeddings in self-attention are derived from the same single embedding. Additionally the transformer uses eight attention heads in parallel, allowing different parts of the input sequences to be attended to simultaneously and thereby capturing diverse relationships and leading to a richer representation of the sequence of word tokens. This architecture, called ‘multi-head attention’ (MHA), outputs to a linear fully-connected layer, as well as residual connections in the encoder. It outputs its contextualised embeddings to another MHA and linear layer in the decoder. The decoder masks each token and predicts it based on the sequence of preceding tokens, itself a MHA layer (Figure 1.12).

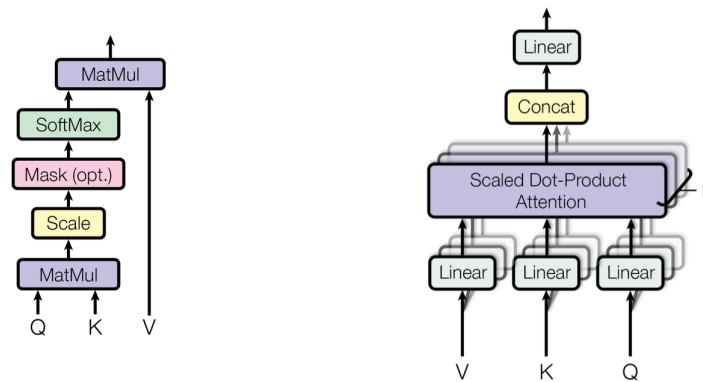


Figure 1.11: Scaled dot-product attention (left) run in parallel (right) (Vaswani et al. 2017).

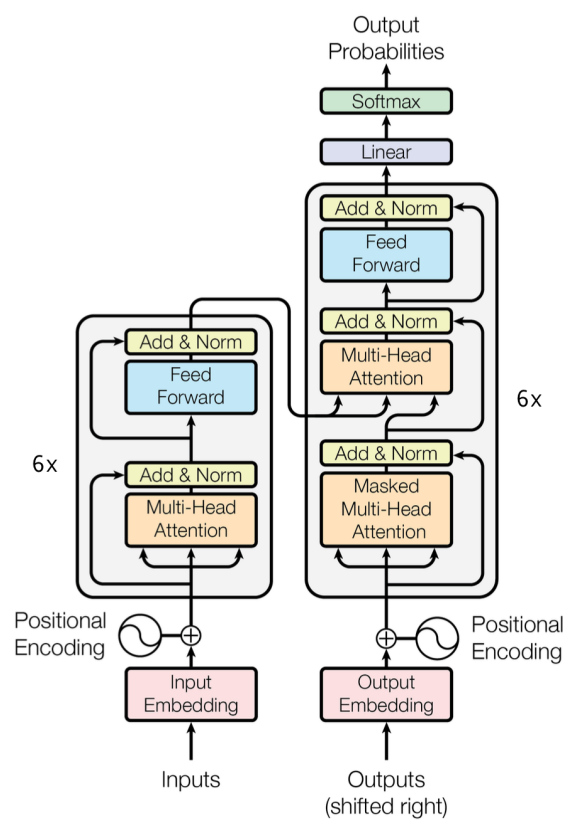


Figure 1.12: The transformer has 6 identical layers of its encoder (left) and 6 identical layers of its decoder (right) (Vaswani et al. 2017).

Generative models and diffusion models

Generative models cover a range of learning systems including hidden Markov models (Baum & Petrie 1966) which have long since been applied successfully to a broad range of problems. Much more recently developed forms of generative models which rely on deep neural networks include most forms of autoregressive models (which might use RNNs and transformers), generative adversarial networks (GANs), normalising flows, variational autoencoders (VAEs) and diffusion models. Of these, normalising flows, VAEs and diffusion models are probabilistic models. Normalising flows are similar to diffusion models in the sense that they define non-linear mappings *from* latent variables to observed data, both quantities bearing the same dimensions. VAEs are similar to diffusion models in the sense that they approximate the data likelihood using a lower bound based on an encoder that maps *to* latent variables. Diffusion models differ from these by virtue of the fact that the encoder is predetermined and the objective is to learn the decoder which is the reverse process, used for generating samples (Prince 2023). GANs excel at image generation, but were surpassed by diffusion models in 2021 leading to greater interest in the latter (Ho, Jain & Abbeel 2020; Dhariwal & Nichol 2021).

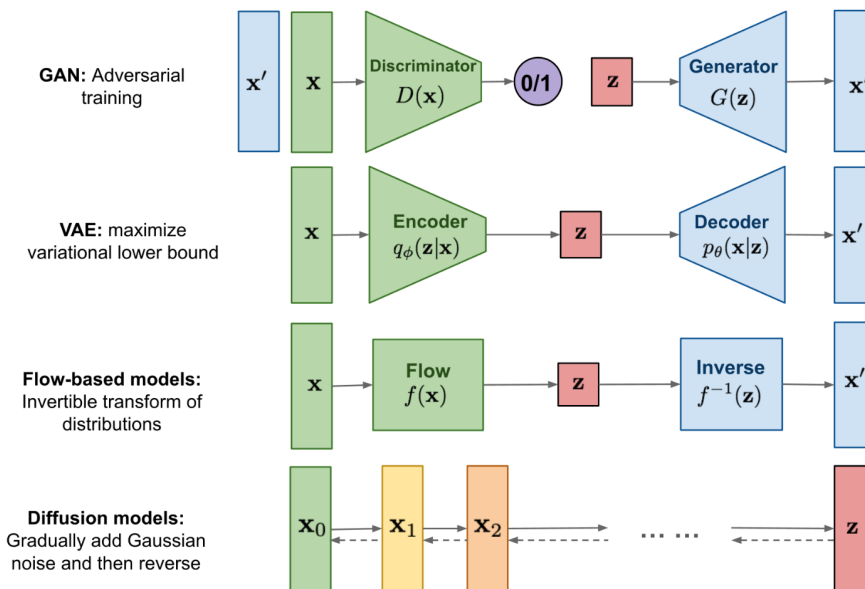


Figure 1.13: Four different generative deep learning models, from top to bottom: generative adversarial networks (GANs), variational autoencoders (VAEs), normalising flows and diffusion models (creative commons).

Diffusion models

The flexibility and potential effectiveness of probabilistic models cannot be fully exploited in machine learning due to the resulting inhibitive computational demands, particularly when dealing with the high dimensional data now prevalent in deep learning. In 2015, a new form of latent variable model, namely the ‘diffusion model’, was published which had a number of advantages over the very many other existing algorithms for learning and inference in the realms of probability distributions (Sohl-Dickstein et al. 2015). It was inspired by similar mechanisms in non-equilibrium statistical physics (Jarzynski 1997); and sequential Monte Carlo sampling (Neal 1998). A diffusion model involves multiple iterations of adding Gaussian noise to input data distributions (equations 1.2 & 1.3, see next page), followed by de-noising back to the original distribution (equations 1.4–1.7). The model aims to learn the parameters of each time step’s distribution (i.e the joint distribution of a latent and time step), one step at a time. The iterations are done as a Markov chain, such that it is also learning a conditional distribution for each step given the previous step. The parameters are learned by a deep neural network.

A number of significant improvements and demonstrations of its application in image synthesis led to greater interest and uptake in the use of diffusion models in image generation tasks (Ho, Jain & Abbeel 2020; Nichol & Dhariwal 2021; Dhariwal & Nichol 2021). Various formulations of the original diffusion model include the denoising diffusion probabilistic models (DDPMs), noise-conditioned score networks (NCSNs) and stochastic differential equations (SDEs) (Song et al. 2020). DDPMs estimate gradients of the data distribution, using a variant of score matching (Ho, Jain & Abbeel 2020). NCSNs describe the same operation but with a re-weighted variant of the evidence lower bound (ELBO) of DDPM (Song & Ermon 2019). NCSNs and DDPMs can be generalised further as stochastic differential equations (SDEs). (I won’t go into further details on these other than to mention that the majority, if not all, diffusion models used in protein design and protein structure-related publications thus far appear to employ a DDPM framework or a ‘score SDE’ framework (Guo et al. 2023)).

Diffusion models

- Simplified forward and reverse steps, based on Sohl-Dickstein et al. 2015

1. Entire *forward process* involves incremental corruption of data $x^{(0)}$ into noise $x^{(T)}$ via Markov chain:

$$q(x^{(0:T)}) = q(x^{(0)}) \prod_{t=1}^T q(x^{(t)} | x^{(t-1)}). \quad (1.2)$$

Where noise in Markov chain progression is Gaussian, one step in forward process is:

$$q(x^{(t)} | x^{(t-1)}) = \mathcal{N}(x^{(t)}; \sqrt{\alpha_t} x^{(t-1)}, \beta_t I). \quad (1.3)$$

where $\sqrt{\alpha_t} x^{(t-1)}$ is Gaussian's mean, (hence $\sqrt{\alpha_t}$ controls scaling of previous state), and $\beta_t I$ is Gaussian's covariance, (hence added noise is isotropic).

2. Entire *reverse process* approximates reconstruction of $x^{(0)}$ from $x^{(T)}$, via reverse Markov process:

$$p_\theta(x^{(0:T)}) = p_\theta(x^{(T)}) \prod_{t=1}^T p_\theta(x^{(t-1)} | x^{(t)}). \quad (1.4)$$

where $p_\theta(x^{(T)})$ is prior distribution at final time step T which is typically just a Gaussian $\mathcal{N}(0, I)$.

Learned reverse transition probability at each time step defined as Gaussian distribution, with parameters learned by model:

$$p_\theta(x^{(t-1)} | x^{(t)}) = \mathcal{N}(x^{(t-1)}; \mu_\theta(x^{(t)}, t), \Sigma_\theta(t)). \quad (1.5)$$

Parameters are trained by minimising the Kullback–Leibler (KL) divergence between the true reverse transition and the learned reverse transition.

$$D_{KL}(q(x^{(t-1)} | x^{(t)}, x^{(0)}) || p_\theta(x^{(t-1)} | x^{(t)})). \quad (1.6)$$

Maximising evidence lower bound (ELBO) as a proxy for minimising the KL divergence.

$$L = \mathbb{E}_q \left[\log p_\theta(x^{(0)}) \right] \geq \mathbb{E}_q \left[\log \frac{p_\theta(x^{(0:T)})}{q(x^{(1:T)} | x^{(0)})} \right]. \quad (1.7)$$

Goal of project

The first part of this project’s goal was to convert a structure prediction diffusion model for RNA molecules (designed and built in-house) into one that takes protein amino acid sequences as inputs and trains a diffusion model on their corresponding atomic structural coordinates. This required implementing a tokenisation protocol followed by testing it on a small dataset, as a proof-of-principle.

The second part of this project’s goal was to train this on a larger dataset, to set benchmarks and perform empirical interrogation of the architecture and hyperparameters, for example by ablation studies, testing different network trunks, as well as heuristic modifications, in order to drive up computational efficiency. This would be similar to the approach taken by (Elnaggar et al. 2023) for protein language model training but on a smaller scale. No similar systematic set of experiments has been explicitly published on diffusion models for protein structure prediction or design.

Unfortunately, only the first part was completed. Fortunately it was completed to a very high standard that lays the ground work for the aforementioned experiments. It was also written in readable modular format to make it accessible for anyone to make modifications or additions to the logic.

Chapter 2

Related Work

Attention models for protein structure

Deeper neural networks perform automatic feature extraction and representation learning (Bengio, Courville & Vincent 2012). The subsequent introduction and adoption of the transformer and related attention models, has led to a gradual migration away from the recurrent neural networks like LSTMs, as well as CNNs. Given sufficient training data, self-attention is inherently more flexible, where the inductive bias is essentially being learned, starting from a fully-connected graph or unordered set, rather than baked in as a 1D grid (as with RNNs) or 2D grid (as with CNNs) (Bronstein et al. 2021), though the transformer’s positional encoding does place some constraints.

For lack of a more mathematical or scientific description, attention models make it easier to learn a “deeper grammar” and to “reason” about the corresponding information in natural language. The importance of incorporating much longer-range contexts within sequences is as informative and important, if not more so, for protein sequences. However, the self-attention calculation can place a large demand on computer memory, so is not always an automatic choice for every protein sequence-structure-function-related deep learning task. Since the release of the transformer, RNNs and CNNs have continued to provide suitable deep learning trunks for specific tasks (Kandathil et al. 2020; Kandathil et al. 2022).

Proteins appear to encode a remarkable amount of hierarchical information within their primary structure. It is no less remarkable that attention models seem to be up to the task of helping to “understanding” this. Perhaps one of the most convincing examples of how effective they can is the self-supervised deep networks of attention models which learn rich sequence embedding, referred to as ‘protein language models’ (pLM) (Rives et al. 2019; Elnaggar et al. 2020). However, even a single layer appears to be capable of learning to predict protein structure from its amino acid sequence (Bhattacharya et al. 2020).

AlphaFold 2

In 2020, a protein prediction neural network achieved significantly better results than competing entries (Kryshtafovych et al. 2021; Jumper et al. 2021). It contained a very deep network containing multiple uses of the attention mechanism, continuing the trend of demonstrating the effectiveness of brute-force computation. The r.m.s.d₉₅ achieved suggests this was approaching the upper limit of possible accuracy for computational prediction of protein structure using only the protein sequence. (The conclusion of this upper limit is based on the level of accuracy one can expect to achieve for structure determination *in vitro*). Following the outcome of this neural network at CASP in 2020, the panel virtually declared protein structure prediction as a solved problem (Kryshtafovych et al. 2019):

“Objective testing in CASP14 has shown that the problem of computing atomic accuracy protein structures from amino acid sequence is solved, at least for single, ordered, proteins.”

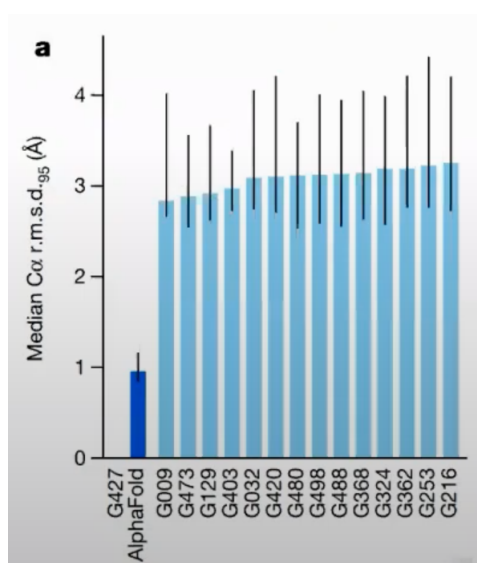


Figure 2.1: Root mean squared deviation (discarding 5% outliers) (r.m.s.d₉₅) in angstroms, shows significantly more accurate structure predictions for AlphaFold than 15 other CASP14 entries (2020) (Figurnov 2021).

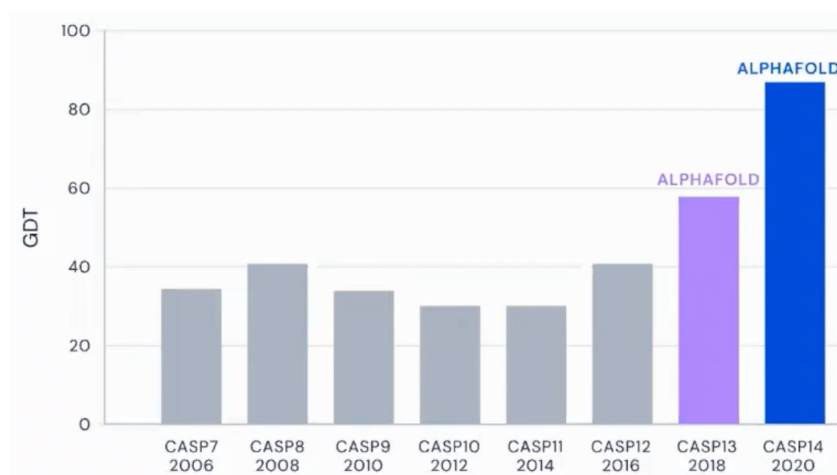


Figure 2.2: Global distance test (GDT) metric showing the relatively large improvement gained by AlphaFold (Figurnov 2021).

AlphaFold 2

There are two inputs: a computationally-expensive MSA on which the accuracy of structure prediction is heavily dependent; and a residue pair representation that encompasses interaction information between residues and can include structural information from homologous proteins of solved structure where available. The combination of MSA attention model representation (including by masking) and the pair representation leads to accurate predictions early in the subsequent network layers of the trunk. This trunk, called ‘Evoformer’ (Figure 2.3), performs iterative graph inference in which the graph edges are pairs of residue side-chain atoms, rather than the polypeptide backbone atoms. The iterative updates refine the graph, aided by transitivity inductive bias like the triangle inequality, which is also processed by attention. The whole network can then be recycled through up to 3 more times, (though backpropagating through only the final cycle), such that Evoformer can be as deep as 192 blocks, leading to improved predictions. A rotationally invariant loss metric was used to train the neural network by gradient descent. This was based on aligning the triangular frame formed by the N-C-C atoms of the backbone of one single residue at a time, and then measuring the Euclidean distance of the ground truth from predicted positions for all the other residues. This loss was designed to be rotation invariant. The loss also considers each residue independently, rather than explicitly as part of a chain. In addition to training a model supervised by ground-truth coordinates of solved structures, a second model makes use of much larger sequence databanks that lack solved structures by using confidently-predicted structures to supervise learning.

Learning the final structural coordinates of the complete protein chain is done by taking the learned residue distance graph, treated as a “gas of 3-dimensional rigid bodies”, and passing it through a separate 8-block network with a 3-dimensional equivariant transformer architecture, thereby completing an end-to-end structure prediction network. Its success supported the idea that where brute-force and very deep networks are an option for researchers, replacing heuristically-defined design decisions with learnable equivalents is preferable in terms of prediction accuracies.

(The term ‘end-to-end’ here indicates that from the input of protein sequence to the output of protein structural coordinates, the gradients of the objective function(s) lead to updates in the learnable parameters throughout *all* hidden layers of the neural network, such that no parts of the pipeline are being optimised according to different objectives.)

Although the architecture in AlphaFold 2 is not directly related to this project, it is included in Related Work because it is considered a major turning point and the pinnacle of deep learning protein structure prediction methods, built upon older methods that are briefly discussed in Introduction. It outperformed the next best deep learning methods like RoseTTAFold (Baek et al. 2021) by a large margin and has strongly influenced the architectural designs and approaches of subsequent efforts in the field as a whole (Robin et al. 2021; Wu et al. 2022; Li et al. 2022; Lin et al. 2022; Baek et al. 2023; Ahdritz et al. 2023). AlphaFold 3, which is discussed in a following section, has many similarities to its predecessor but employs a diffusion model, making it more directly relevant here.

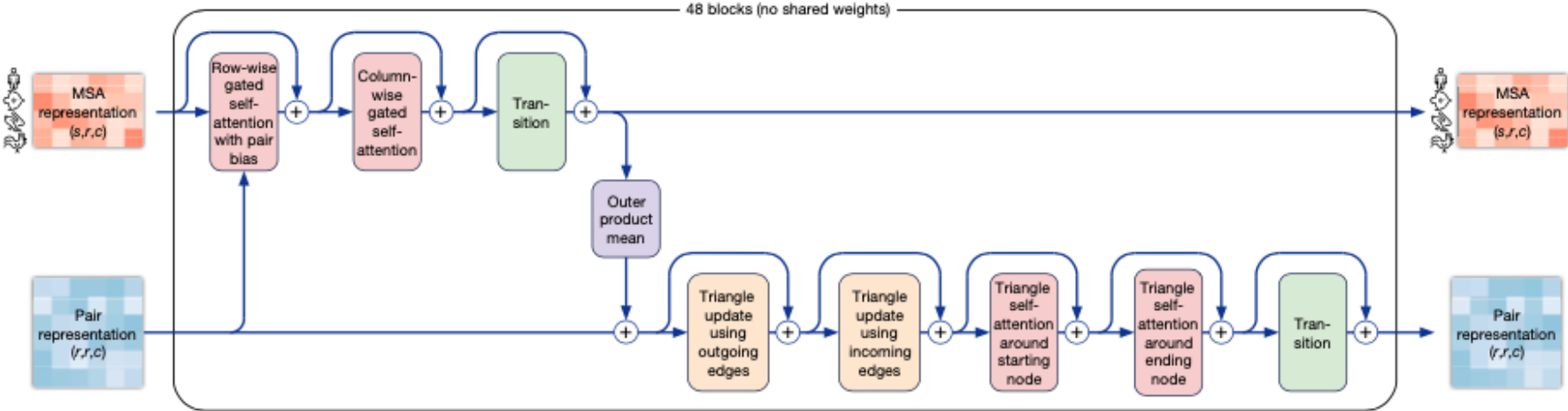


Figure 2.3: Evoformer trunk of AlphaFold 2 (Jumper et al. 2021).

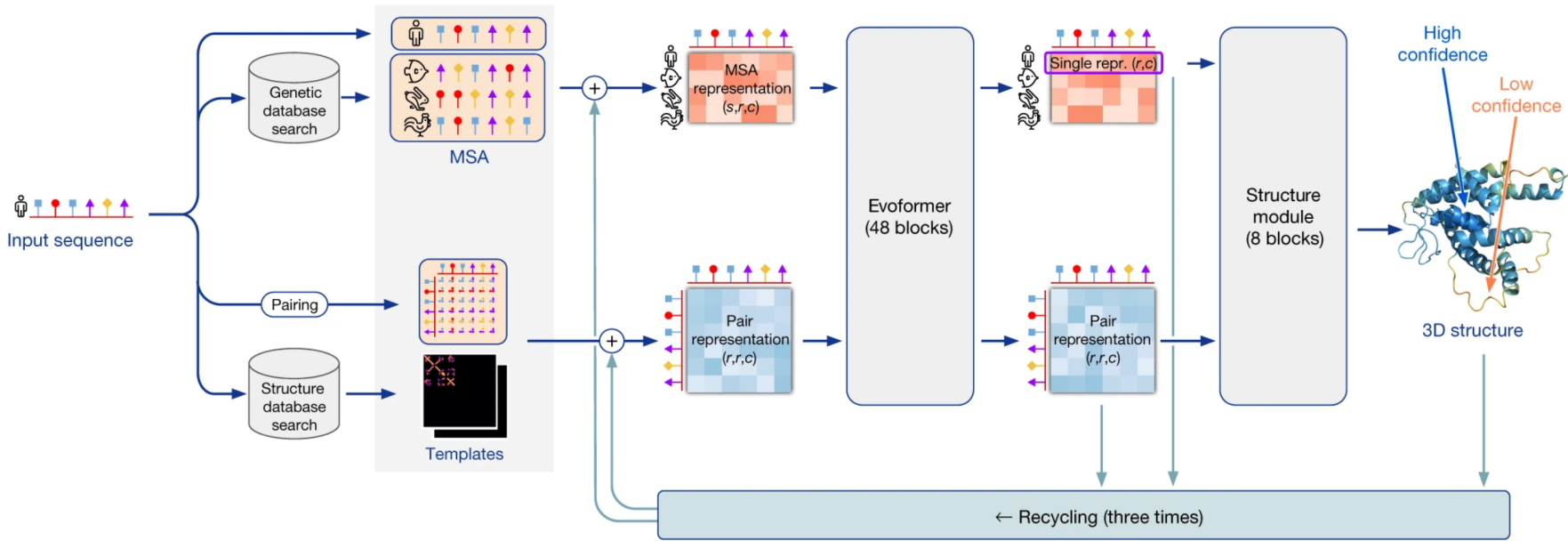


Figure 2.4: AlphaFold 2 (Jumper et al. 2021).

Embeddings and protein language models

Vector-based embeddings (Hinton 1986) are a standard representation format of natural language in statistical NLP (Mikolov et al. 2013). Natural language embeddings describe the numerical representation of a word embedded in vector space. Prior to deep learning innovations, a number of variants of Lipschitz embeddings were used with ‘biological language’, designed to enable quantifying dissimilarity between protein sequences (Hjaltason & Samet 2003). These were partly motivated by the goal of improving computational efficiency in the face of large and changing datasets, (rather than an unbiased extraction of latent biophysically-meaningful information). They also struggle to satisfy the triangle inequality in vector space. These were superseded by developments in NLP, whereby embeddings generated by models that learned statistical distribution of word sequences. Similarities of natural language to biological sequences inspired the investigation of generating embeddings using similar methods (Asgari & Mofrad 2015; Yang et al. 2018; Mazzaferro 2017). These were generated by shallow neural networks and were of relatively low dimensionality. As with the more powerful protein language models (pLM) discussed below, these early models were recognised as serving the role of base models to be used in transfer learning and fine-tuning to specific downstream tasks. These began to be superseded by other self-

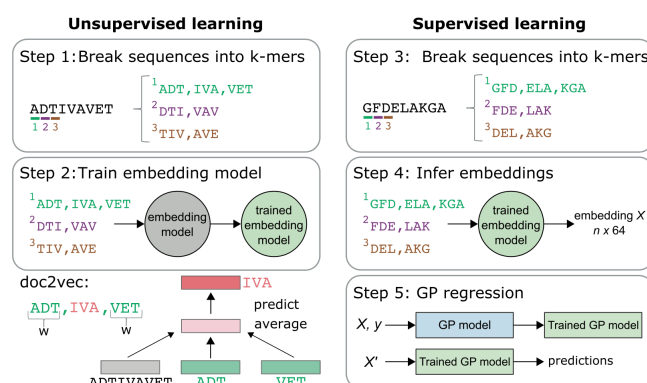


Figure 2.5: Generation of embedded representation of by self-supervised machine learning method (left) (Yang et al. 2018) based on doc2vec. Right panel represents transfer learning in which the embedding is used for downstream supervised learning tasks.

supervised models that were generated by deep neural networks and produced embeddings with higher dimensionality. A significant step up in natural language understanding came from the use of deep neural networks of LSTMs which could generate contextualised embeddings of each word token by incorporating the surrounding tokens (Peters et al. 2018). Such embeddings, created by self-supervised learning, could encode not only syntax but semantics and polysemy. Once again, an advance in NLP was tested on protein language learning, this time via training deep LSTM-based architectures on protein sequences from the large unlabelled dataset, UniRef50 (Heinzinger et al. 2019). The learned knowledge in these trained embeddings were built upon with subsequent task-specific training, for example with small CNNs. Despite the growing popularity and uptake of attention models, recurrent and convolutional models have continued to feature in neural networks studies including for training embedding models (Rao et al. 2019; Rives et al. 2019). Regardless of the choice of architecture, the emergence of the more recent pLM embeddings has emerged somewhat in parallel and separately from the aforementioned ground-breaking applications of other brute-force deep learning models (i.e. AlphaFold), which rely heavily on multiple sequence alignments (MSAs). Here, there is no explicit MSA being performed and this is a very significant difference. Generating an MSA is very computationally-demanding and can take a relatively long time to complete.

Embeddings and protein language models

As the statistics of word proximity encode semantics in corpora (Harris 1954), so too are the statistics of residue proximity observed to encode structure (and function) in proteins (Yanofsky, Horn & Thorpe 1964). The importance of longer-range interactions is undoubtedly relevant in proteins, given that most fold, so that there are multiple interactions between residues at distal sites in the primary structure. Their combinations, composed of different physicochemical interactions and bond formations, have the potential of being highly complex. This might amount to the type of “deep grammar” that attention models might be better suited for extracting than other architectures like RNNs and CNNs. Indeed a single attention layer alone was found capable of predicting contact maps (as already mentioned in Introduction) (Bhattacharya et al. 2020; Rao et al. 2020).

Protein language model - evolutionary scale modelling (ESM)

Application of attention models to 250 million sequences of the Uniparc database (The UniProt Consortium, 2007) was performed (Rives et al. 2019). In a comparison of different architectures, the best results were found to be produced by transformers and those with the deepest network (34 or 36 layers), as well as with the largest number of parameters (669 million).

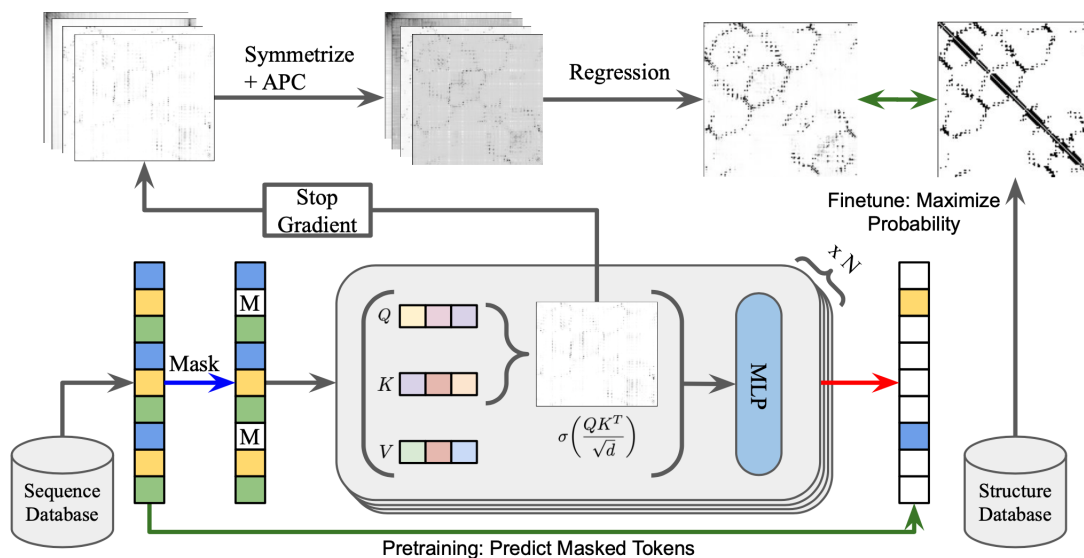


Figure 2.6: Evolutionary scale modelling (ESM) protein language model (Rao et al. 2020).

A second major version of ESM was developed, with an improved architecture and trained using masked language modelling objective (Lin et al. 2022). Continuing on the trend that brute-force computation is beneficial, it was found that a large increase in the number of parameters (up to 15 billion) produced improved resolution in structure prediction. This was trained on 65 million protein sequences from UniRef (Suzek et al. 2015). When this pLM was combined with a folding trunk, designed to replicate the Evoformer in AlphaFold 2 (Figure 2.3), adapted to apply attention mechanisms to a sequence rather than an alignment of sequences, the result was a protein structure prediction network that was approaching the accuracy of AlphaFold 2, while being far more computationally efficient than AlphaFold 2.

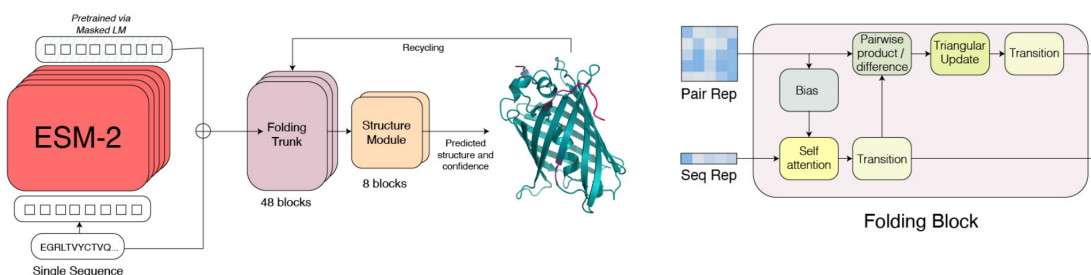


Figure 2.7: ESMFold structure prediction using version 2 of ESM protein language model. The folding trunk/block is shown in greater detail on the right (Lin et al. 2022).

Protein language model - evolutionary scale modelling (ESM)

The third and most recent version of the ESM pLMs is trained on a larger number of parameters still (98 billion) (Hayes et al. 2024). Its stated use is shifted away from being entirely about representation learning to have generative utility, enabling the *ab initio* design of new protein structures, rather than purely providing an embedding that aids protein structure prediction via transfer learning. As with the first two versions, it undergoes self-supervised training by a masked modelling method. (The rate of masking influenced whether the model was better for generative applications or representation learning). Unlike the previous versions, ESM3 is self-trained on a variety of different modalities of tokens instead of just amino acid sequence tokens. The architecture is a bidirectional stack of transformer blocks and includes one with geometric attention. It is not clear if this geometric attention block is different from the “equivariant transformer” mentioned in the ESM2 publication (Lin et al. 2022). (It may simply have been a reference to attention on pairwise distances, which are invariant to rotation and translation). There are 7 types of input tokens in total that combine into an embedding, while structure coordinates are directly input to the geometric attention block.

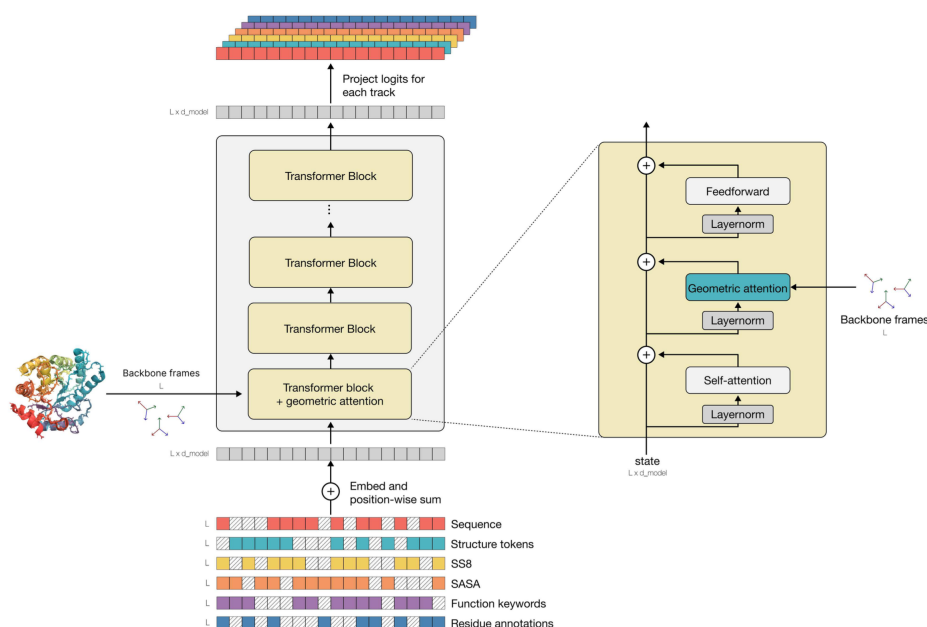


Figure 2.8: Version 3 of ESM protein language model (Hayes et al. 2024).

The inclusion of ESM3 in Related Work is partly for completion as the most recent addition to the family of ESM models. It is also representative of the more recent trend of shifting focus away from predicting 3D structures of single domain proteins, towards generative models that attempt to perform protein design, docking and binding. One of these models will be briefly described in the

Protein language model - ‘ProtTrans’

The aforementioned ESMs demonstrated how very large scale protein language models (pLM), trained using transformers, could be very effective at representation learning in protein sequences for structure prediction. In a similar vein, contextualised embeddings of amino acid sequences using attention-based models had also been explored in separate research that began by comparing the performance of six different pre-existing transformer architectures (Elnaggar et al. 2020). These included the very widely-used model BERT (Devlin et al. 2019) (which is an encoder-only transformer) and T5 (Raffel et al. 2019) (which is an encoder-decoder transformer). These were trained on either UniRef50 (45 million proteins) (Suzek et al. 2015), UniRef100 (216 million proteins), BFD (2 billion proteins) or made use of pairs of these. The comparison was made by taking the last hidden state of the transformer’s attention stack and using this as input to a supervised CNN for example, in order to perform downstream protein-related prediction tasks including for structure. As such, the performance of the second part served as a proxy of how well the different transformers had learned and encoded relevant representations into the embeddings. The T5-based models were reported to be the most accurate at predicting secondary structure in the most recent CASP dataset. It was also shown that accuracy improved if the model was pretrained first on the BFD dataset and then fine-tuned on the UniRef50 dataset. In this task the best model was reported to be marginally more accurate than the best of the ESM1s (ESM1b).

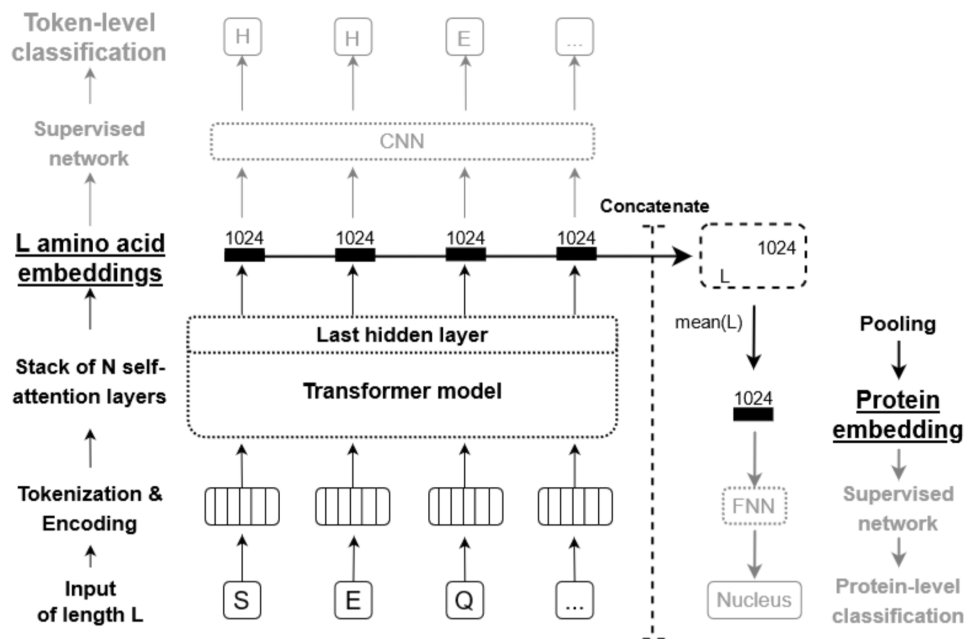


Figure 2.9: Schematic of the ProtTrans protein language model pre-training and downstream transfer learning (to protein-level or token-level classification) (Elnaggar et al. 2020).

Protein language model - ‘Ankh’

Despite previously reporting (with ProtTrans) that, on the whole larger training datasets, larger models and longer training produced greater accuracies, this was directly challenged in a follow-up study focused on systematically interrogating ways at improving computational efficiency (Elnaggar et al. 2023). Their model is an encoder-decoder transformer (with 48 encoder layers and 24 decoder layers), trained by self-supervised masked modelling method. They present two sizes of this model: one with 450 million parameters, 12 attention heads and 3072 feed forward layers; the other with 1.15 billion parameters, 16 attention heads and 3840 feed forward layers. Their performances are compared over a variety of 7 downstream tasks, 3 of which are explicitly structure-related, (though the other 4 may be indirectly related to structure too). There is some conflicting data in the paper which appears to indicate the larger of their two models performing marginally better on average (Fig 1a of paper), but appears to contradict values they present in a table (Table 1 of paper). Nonetheless, their model (of either size) was optimised after performing a host of different experiments interrogating architecture, hyperparameters, activation functions, masking probability, pretraining datasets, amongst others. Each of these displayed relatively marginal effects in most cases, but in combination resulted in a more computationally-efficient pLM, that was made freely-available for other researchers to use.

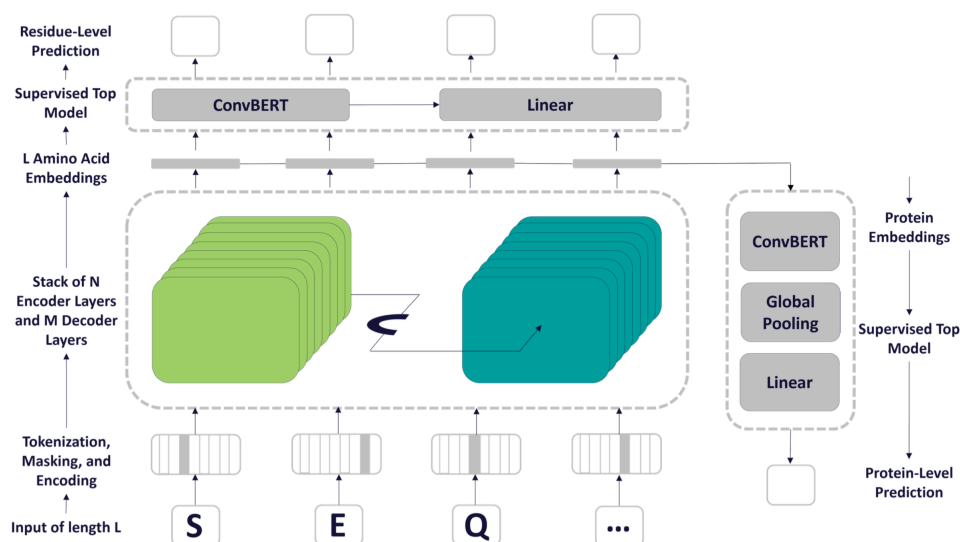


Figure 2.10: Schematic of Ankh protein language models and the downstream supervised transfer learning prediction tasks (residue-level or protein-level) (Elnaggar et al. 2023).

The smaller version of the model, Ankh-base, is the pLM used to generate embeddings in this MSc project. The approach and rationale of testing a wide range of adjustments is in line with that intended originally as the second part of the goal for this MSc project.

Diffusion models for protein structure

In terms of the current literature diffusion models alone have not been applied to protein structure prediction *ab initio*. They have, however, been used for generating native-like protein sequence-structures pairs *de novo* (aka ‘protein design’)(Anand & Achim 2022; Trippe et al. 2023; Watson et al. 2023; Lee, Kim & Kim 2023; Ingraham et al. 2023; Wu et al. 2024; Alamdari et al. 2024; Chu et al. 2024), as well as protein ligand-binding (Abramson et al. 2024) and molecular docking (Corso et al. 2022).

The potential utility of deep probabilistic generative models for designing modified or relatively small novel proteins was demonstrated not long ago with variational autoencoders (VAEs) (Greener, Moffat & Jones 2018)) and generative adversarial networks (GANs) (Anand & Huang 2018). Soon after publication of improvements (and open-source code) to diffusion model frameworks (Ho, Jain & Abbeel 2020; Dhariwal & Nichol 2021; Song & Ermon 2020) their applications to protein structure began to emerge. Some of these are described briefly in the following pages.

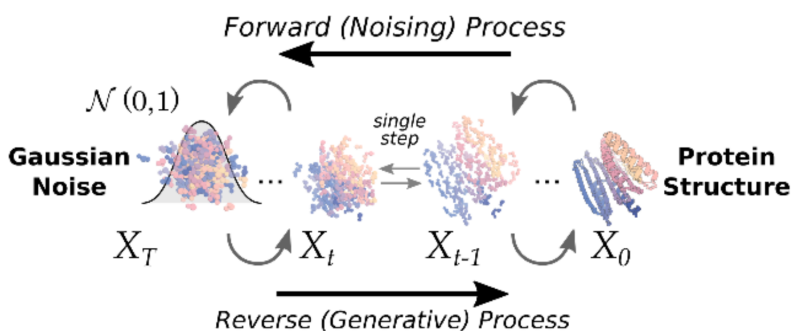


Figure 2.11: Schematic of protein structure diffusion model (Watson et al. 2023).

Diffusion models for protein structure

Anand & Achim trained a diffusion model on experimental data and conditioned on a topology prior (Anand & Achim 2022). The prior being a coarse encoding scheme involves secondary structure-based heuristics and alpha-carbon distance matrix, embedded via triangular self-attention network. All-atom structures of “realistic” *de novo* proteins were generated across the full range of domains in the protein data bank, including larger multi-domain proteins. (The coarse encoding scheme allowed for generating more than one structure). Sequences were then generated by a form of masked language model. Equivariance was addressed by using a loss function that incorporated frame-aligned point error (FAPE) while the main deep learning trunk was composed of invariant point attention (IPA).

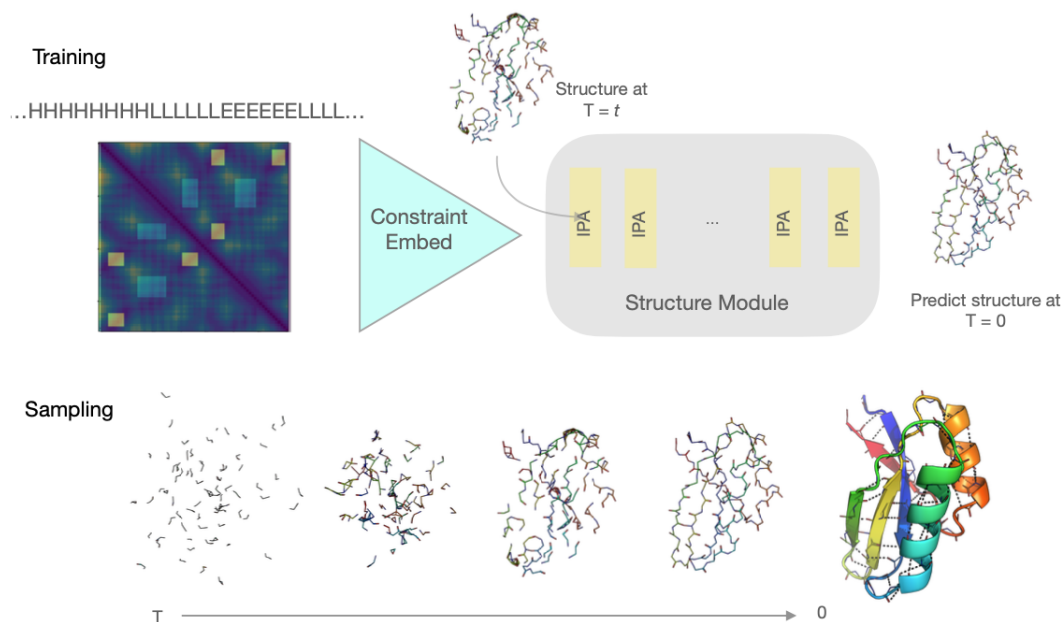


Figure 2.12: Blocks of invariant point attention (IPA) learn to denoise the diffuse structure (top) with inputs of constraints: a secondary structure-informed heuristic (top-left) and C_α distance matrix embedded via triangular self-attention before processing (Anand & Achim 2022).

Lee et al. trained a diffusion model (SDE) using an image-like representation of the backbone using 6D coordinates (Lee, Kim & Kim 2023), according to trRosetta (Yang et al. 2019). This addressed the requirement for equivariance considerations. Once trained, 6D backbone coordinates are unconditionally generated and processed to produce a full-atom model via four more protein structure programs/algorithms.

Wu et al. trained a diffusion model on a BERT-style transformer using proteins represented as backbone torsional angles (ϕ , ψ , ω) for pairs of adjacent residues (Wu et al. 2024). Hence the model predicts the 6 angles between residues rather than the absolute positions of atoms. This allowed the absence of any other form of equivariance considerations. The trained model achieved unconditional generation of realistic protein structures, though the majority were found to be undesignable.

Diffusion models for protein structure

Lin & AlQuraishi trained a diffusion model upon proteins represented as a cloud of points (during forward noising) or oriented reference frames (coordinates) in 3D space (during sample generation) (Lin & AlQuraishi 2023). As in Anand & Achim 2022, the model utilises invariant point attention (IPA), satisfying SE(3)-equivariance requirements. The model generates backbone atoms for novel but ‘designable’ proteins.

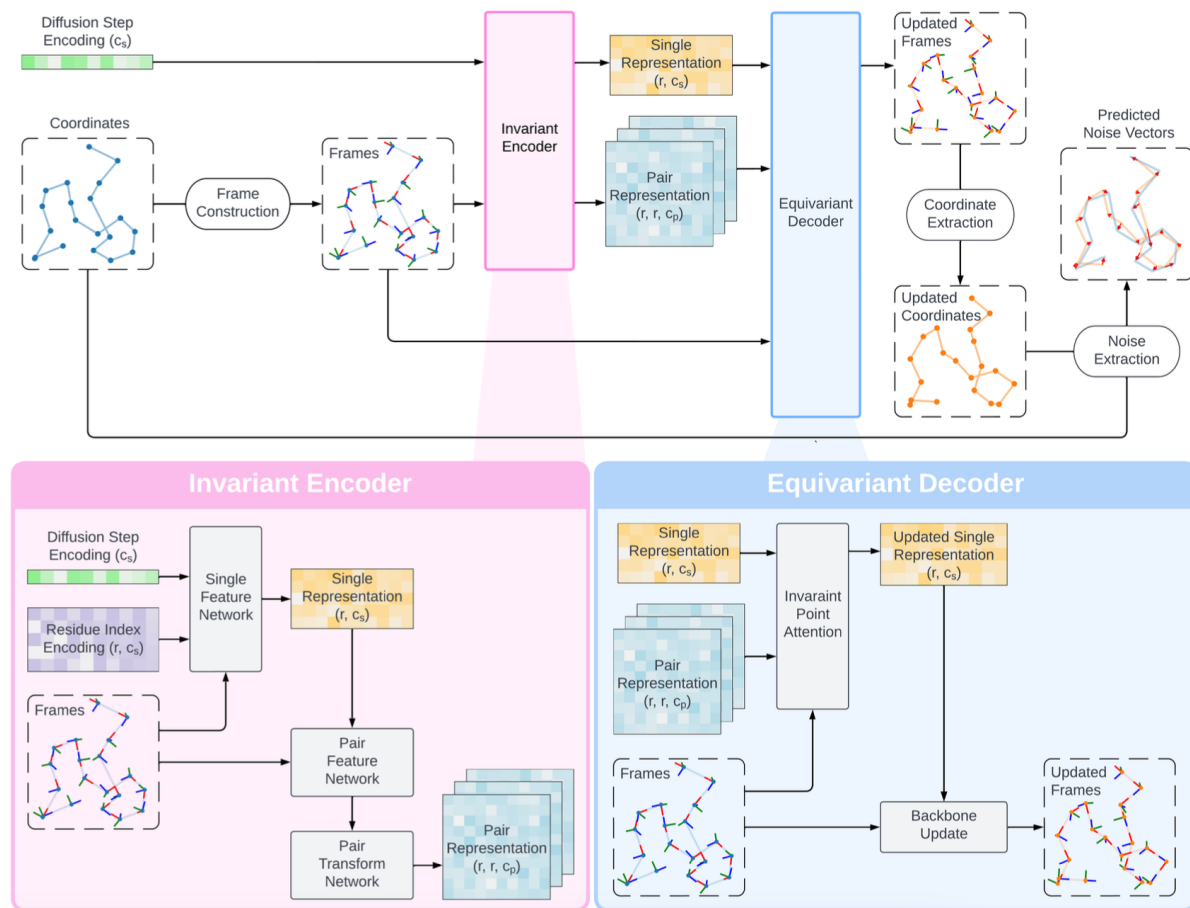


Figure 2.13: The complete architecture of protein design diffusion model. Noising is performed by the encoder, denoising by the decoder (Lin & AlQuraishi 2023).

Watson et al. demonstrated a diffusion model could be trained to generate all-atom protein structure by fine-tuning a pre-trained protein structure neural network model, specifically RoseTTAFold (Baek et al. 2021; Watson et al. 2023). Protein structures were represented in a manner consistent with RoseTTAFold, namely the C_α atom coordinates and N- C_α -C rigid orientation per residue. The loss function was the mean-squared error loss between frame predictions and ground truth protein structure was used without alignment and averaged over all residues. It inherits its equivariance from RoseTTAFold. The trained model was capable of auxiliary conditioning information, such as partial sequence and fold information, or motif coordinates. Performance in all tasks were improved when the model was trained by a ‘self-conditioning’ strategy in which predictions could be modelled on predictions at previous time steps. Amino acid sequences for the structures were generated by a separate pre-trained network (Dauparas et al. 2022) and the sequence-structure combination validated by AlphaFold 2 and proteins synthesis *in vitro* confirmed the *de novo* designs (Torres et al. 2023).

Diffusion models for protein structure

Yim et al. constructed and trained an SE(3)-equivariant diffusion model to generate novel protein backbones (Yim et al. 2023). The model is trained to noise and denoise backbone residues represented as solid frames (Figure 2.14), using invariant point attention (IPA) and transformer neural networks. The diffusion process is centred at the origin to ensure translation invariance and noise is isotropic ensuring rotational invariance.

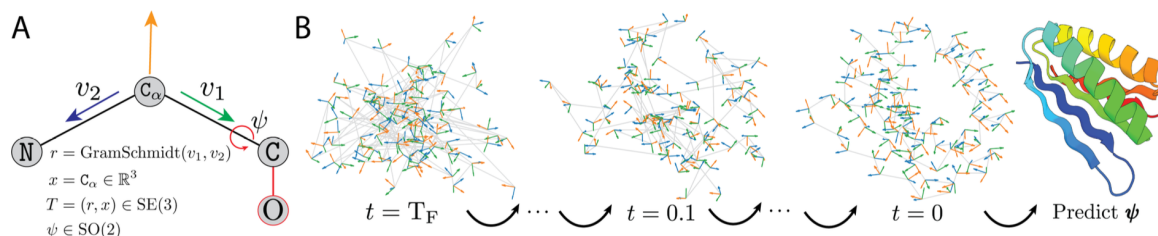


Figure 2.14: Frame representation of a backbone residue (A, left). Denoising process (B, right) (Yim et al. 2023).

In a large study Ingraham et al. trained a diffusion model on a novel GNN, in order to generate backbone atom coordinates of novel proteins (Ingraham et al. 2023). Their pipeline included a design network with another GNN for taking the generated backbone and generating an amino acid sequence and side-chain atoms. The sampling was designed to be able to be conditioned to produce proteins with desired properties or functions, guided by external constraints like symmetries, substructure, shape, semantics and even natural-language prompts. Many of the proteins were synthesised *in vitro* with experiments supporting the viability of this protein design method.

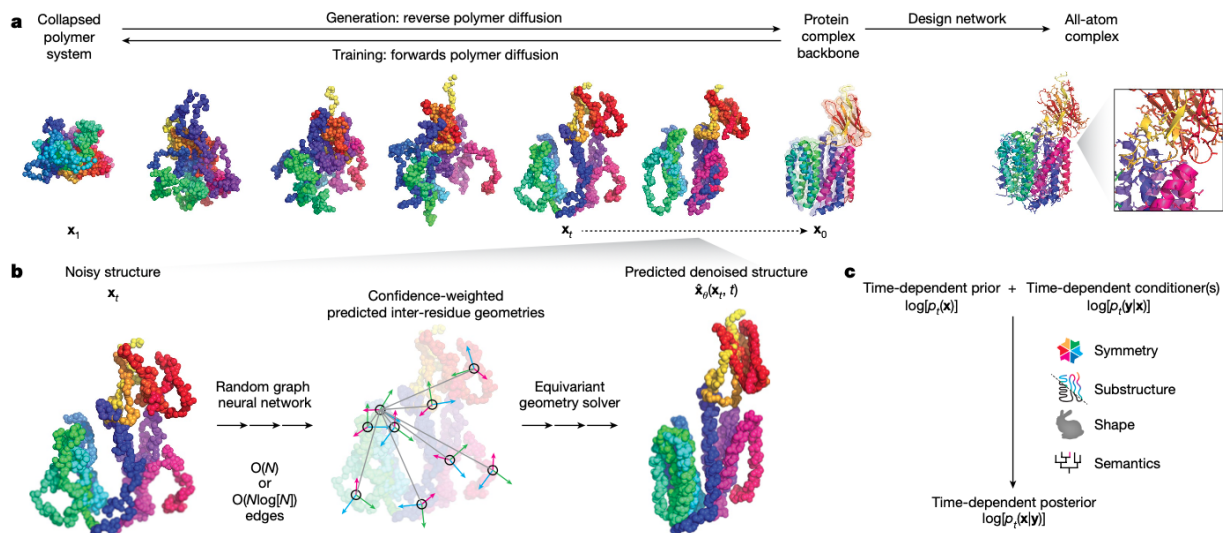


Figure 2.15: Schematic representing overall approach of noising and denoising protein backbone coordinates, followed by a design GNN that generates an amino acid sequence and side-chain coordinates for the generated backbone (Ingraham et al. 2023)

Diffusion models for protein structure

Chu et al. trained a diffusion model on proteins as 3D point clouds of coordinates, using U-ViT module (Chu et al. 2024). As for Yim et al., the diffusion process is made invariant to rotations by applying isotropic Gaussian noise and invariant to translation by moving the centre of mass for all protein structures to the origin. They ensure that despite not using an equivariant loss function or network architecture, the model is sufficiently SE(3)-equivariant due to the data augmentation method that provides the training data with randomly rotated copies. They make the argument that making the network fully SE(3)-equivariant by replacing the U-ViT with an equivariant module is not actually necessary for their model and would only otherwise add extra computational demand. During structure diffusion, a smaller version of a structure-conditioned sequence generating network (used in Watson et al.) was capable of generating the amino acid sequence sufficiently well. Another unique aspect of this study was to enable the model to reason over side-chains jointly with backbone atom positions. This was partly achieved by modelling side-chains as a ‘superposition’ of all possible 20 amino acids, collapsing upon sampling or update.

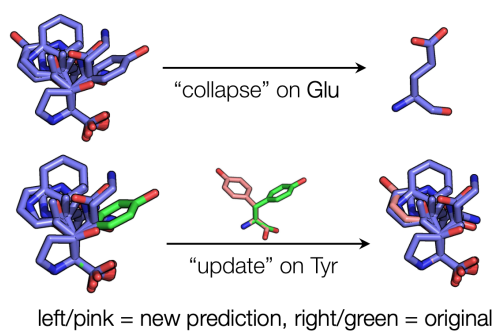


Figure 2.16: Representation of side-chain ‘superposition’ strategy to enable reasoning about side-chains at same time as backbone atoms during the diffusion process. Collapse to single residue occurring during sample generation, or updated for a given amino acid (Chu et al. 2024).

Diffusion models for protein structure

The third version of AlphaFold has many similarities to its predecessor but is ultimately a generative model, having replaced the ‘structure module’ with a ‘diffusion module’ at the top of the neural network (Abramson et al. 2024). The diffusion module is conditioned on the input sequence, pair features and single representation (Figure 2.17). Other differences, of less relevance for this diffusion model section but mentioned for completion are: the row-wise and column-wise attention that received the MSA in AlphaFold 2 are removed. The MSA representation is passed through only 4 blocks instead of all 48 blocks of the trunk, such that the main 48 block trunk takes pair representation and single sequence representations, making it faster and more streamlined (renamed to ‘Pairformer’).

The focus of AlphaFold 3 is shifted to include the binding (or docking) of protein to other proteins or to non-protein molecules such as lipids, polynucleotides. The success of the first two versions at CASP13 and CASP14 suggests this third version might also fair well, though this is not clear at present.

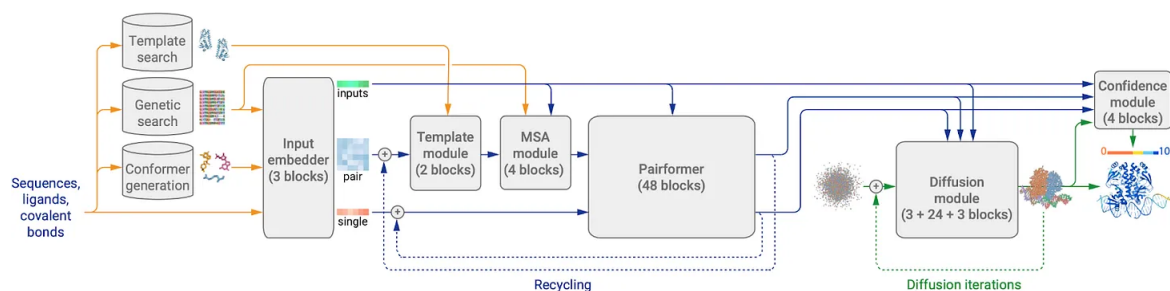


Figure 2.17: AlphaFold 3 (Abramson et al. 2024).

The diffusion module takes Pairformer’s output embeddings as its input. Unlike most of the aforementioned studies using diffusion models for protein design, the diffusion module in AlphaFold 3 is being used to generate the full atomic structure, and as part of the protein structure (and protein-ligand) prediction for a given protein sequence, rather than *de novo* protein design. In most of the other studies, the diffusion model part is responsible for generating a backbone skeleton, which must then be passed over to some separate design modules to provide side-chains and sequence. Another important distinction is in the enforcement of equivariance. The authors of AlphaFold 3 reported that rotational and translational equivariance did not need to be explicitly enforced, and consequently was omitted to reduce computational load. (Although, it is noted that training datasets were augmented with randomly rotated samples which may implicitly incorporate equivariance). The diffusion module therefore operates on tokenised sequence inputs and raw atomic coordinates without explicit equivariant representations. The role of the diffusion module here in *ab initio* structure prediction is more challenging in the sense that the target is likely a single very specific 3D structure, whereas in unconditional *de novo* protein design, any native-like structure is essentially a successful outcome.

Chapter 3

Methods

Software

- All programs are written in Python (version 3.12)
- Version control: https://github.com/zibae-2021/MSc_2024_project
- Package and environment management: Conda
- Dependencies:

Package	Version (macOS)	Version (Rocky Linux)
miniconda	24.11.2	24.11.0
python	3.12.3	3.12.2
numpy	1.26.4	2.0.1
pandas	2.2.3	2.2.3
biopython	1.84	1.84
requests	2.32.3	2.32.3
yaml	0.2.5	0.2.5
matplotlib	3.10.0	3.9.2
transformers	4.48.1 (pip install transformers -U)	4.46.3 (conda-forge)
pytorch	NA	2.5.1 (py3.12_cuda12.1_cudnn9.1.0.0)
torchvision	NA	0.20.1
pytorch-cuda	NA	12.1
einops	NA	0.8.0

Table 3.1: Package versions on the two machines used for development and training.

Package	Version	Build	Channel
cuda-cudart	12.1.105	0	nvidia
cuda-cupti	12.1.105	0	nvidia
cuda-libraries	12.1.0	0	nvidia
cuda-nvrtc	12.1.105	0	nvidia
cuda-nvtx	12.1.105	0	nvidia
cuda-openssl	12.6.77	0	nvidia
cuda-runtime	12.1.0	0	nvidia
cuda-version	12.6	3	nvidia

Table 3.2: CUDA package versions, builds, and channels (on Rocky Linux machine).

- GPU: NVIDIA GeForce GTX 1080 Ti (Rocky Linux machine)

Directory structure:

```

project
├── data_layer
│   └── data_handler.py <----- [ general read-write utility functions ]
├── launchers
│   ├── 1_run_tokeniser.sh
│   ├── 2_build_embeddings.sh
│   ├── 3_train_model.sh
│   └── tokenise_embed_train.sh <----- [ launch full pipeline from here ]
├── rnatrain <----- [ original RNA structure prediction pipeline ]
├── src
│   ├── diffusion
│   │   ├── diff_data
│   │   │   ├── emb
│   │   │   │   └── (.pt files) <----- [ pLM embeddings ]
│   │   │   ├── mmCIF
│   │   │   │   └── (.cif files) <----- [ raw mmCIFs ]
│   │   │   ├── PDBid_list
│   │   │   │   └── (.lst files) <----- [ lists of PDBids ]
│   │   │   └── tokenised
│   │   │       └── (.ssv files) <----- [ tokenised mmCIF data ]
│   │   ├── pt_files
│   │   │   └── (.pt model weights)
│   │   ├── dataset_loader.py
│   │   ├── nn_def_protfold_atomp2.py <----- [ neural network definitions ]
│   │   ├── pytorch_protfold_allatomclustrain_singlegpu.py <----- [ model trainer ]
│   │   └── tokenise_embed_train.py
│   ├── losses
│   ├── pLM
│   │   └── plm_embedder.py <----- [ pLM embeddings builder ]
│   └── preprocessing_funcs
│       └── tokeniser.py <----- [ mmCIF tokeniser ]

```

Figure 3.1: Directory structure showing only the most important folders, scripts and data files.

(For the purposes of facilitating user-testing and grading, some data files (e.g. `.cif`, `.ssv` and `.pt`) are added to git, so these are visible on Github (https://github.com/zibae-2021/MSc_2024_project), though I realise version control is not designed for data files).

Deep learning pipeline - summary:

- `tokeniser.py` parses and tokenises mmCIF, saving each one to `.ssv` file (pages 41-49):
 - Read in specified mmCIF files,
 - Parse mmCIF files.
 - Tokenise mmCIF files.
 - Save mmCIF files to flat files (`.ssv`).
- `plm_embedder.py` generates embeddings, reading protein sequences from `.ssv` files (page 50):
 - Read in tokenised mmCIF `.ssv` files.
 - Extract PDBid_chains and amino acid sequence from `.ssv` files.
 - Generate Ankh-base pLM embeddings for each amino acid sequence.
 - Save embeddings to `.pt` files.
- `pytorch_protfold_allatomclustrain_singlegpu.py` trains the model (pages 51-55):
 - Load training and validation datasets via `dataset_loader.py`.
 - Load and augments datasets into `DMPDatasets`.
 - Update model weights in training loop:
 - Perform forward pass of neural network.
 - Calculate the loss.
 - Calculate partial derivatives of loss wrt trainable parameters.
 - Update parameters (i.e. model weights) and embeddings.
 - Save model weights to `.pt` files.

Building a small training dataset:

The entire list of domain ids from the latest official version from the file `cath-domain-list.txt` was manually downloaded from CATH webserver (Orengo et al. 1997) at: http://download.cathdb.info/cath/releases/all-releases/latest_release/cath-classification-data/cath-domain-list.txt and saved to `data/dataset/big_files_to_git_ignore/CATH/cath-domain-list.txt` (which I haven't added to `git` but is on 'joe-desktop' lab machine).

Starting then from 500,238 proteins (and 12 columns of data fields), the dataset is filtered by process of elimination to just 573 proteins, as follows:

- Removing records for NMR structures, obsolete structures & low resolution X-ray structures.
- Keep only proteins of classes 1, 2 or 3.
- Remove data fields that are related to sequence identity.
- Store PDBids in new column, which helps filter in those with only one domain.
- Filter in PDBs with unique 'Architecture', 'Topology' and 'HomologousSF' values.

This results in reducing number of PDBs from 28,496 proteins to only 573, but this provides a diverse dataset and serves well as a 'dummy' dataset.

- The PDB_Id, DomainID, Architecture, Topology, HomologousSF, Domain_len, Angstroms fields of this 573 single-domain diverse set of proteins PDBids is saved to `data/dataset/big_files_to_git_ignore/CATH/SD_573_CIFs.csv`.

Parsing & tokenising mmCIF

Parsing and tokenising mmCIF is performed in a separate precursor step to the subsequent model training. Files are stored in human-readable flat files (currently `ssv` making manual inspection easy. Doing the tokenisation separately also reduced unnecessary repeat computations and aided development of the code, which did not require CUDA to run.

- **Brief description of mmCIF:**

Macro-molecular Crystallographic Information Files (mmCIF) were designed to replace the original and much smaller and simpler Protein Data Bank (PDB) files. They contain much more protein structure-related data, in a nested hierarchical format with groups of related data in ‘categories’ of ‘attributes’. Attributes from different categories hold duplicate data, enabling cross-referencing between and therefore joining of different categories - so that mmCIF is essentially a graph of data attributes and categories. There are over 200 categories in an mmCIF, but only two of them were needed here, specifically (`_pdbx_poly_seq_scheme` and `_atom_site`).

- **Running the parser-tokeniser script:**

The parser-tokeniser is run by `python3 tokeniser.py` in the activated `conda` environment, assuming your current working directory is `src/preprocessing_funcs`.

This runs `parse_tokenise_write_cifs_to_flatfile()` from the `if __name__ == '__main__':` block. The user should consider the options within this block and comment in/out accordingly. The default choice of which mmCIFs to parse-tokenise is based on what mmCIF files are already downloaded to `src/diffusion/diff_data/mmCIF`. Furthermore, if any of the corresponding `ssv` files are found in `src/diffusion/diff_data/tokenised`, it indicates that these mmCIFs have already been parsed and tokenised previously and will not be repeated. If the user wants to parse-tokenise all mmCIFs afresh, then `dh.clear_diffdata_tokenised_dir()` (on line 778) should be uncommented. Apart from this default setting, the user has the options to pass in a different relative mmCIF directory path, and/or the relative path of 1st file of PDBids or PDBid_chains e.g.

`../diffusion/diff_data/PDBid_list/pdbchains_565.lst`, and/or a Python string or list of PDBid(s) or PDBid_chain(s), e.g. `'1ECA_A'` or `['1ECA_A', '2DN1']`, `['1ECA', '2DN1']`. The function will combine all of these and proceed to download, parse and tokenise them as described in the following subsections.

- **Downloading mmCIF and reading into Python dicts:**

`tokeniser.get_mmCIF_data()` uses a well-established, actively-maintained open-source package called Biopython to read each mmCIF into a Python dict data structure. Any pre-downloaded mmCIFs are read in from `src/diffusion/diff_data/mmCIF` otherwise retrieving them via `api_caller.call_for_cif_with_pdb_id(<PDBid>)` which uses `https://files.rcsb.org/download/<PDBid>.cif`.

Parsing & tokenising mmCIF

- **Example snippet of `_pdbx_poly_seq_scheme`:**

This category has 12 attribute names, listed vertically from the top down. Their 12 corresponding attribute values are below the names arranged horizontally like columns in tabular format.

```

_pdbx_poly_seq_scheme.asym_id
_pdbx_poly_seq_scheme.entity_id
_pdbx_poly_seq_scheme.seq_id
_pdbx_poly_seq_scheme.mon_id
_pdbx_poly_seq_scheme.ndb_seq_num
_pdbx_poly_seq_scheme.pdb_seq_num
_pdbx_poly_seq_scheme.auth_seq_num
_pdbx_poly_seq_scheme.pdb_mon_id
_pdbx_poly_seq_scheme.auth_mon_id
_pdbx_poly_seq_scheme.pdb_strand_id
_pdbx_poly_seq_scheme.pdb_ins_code
_pdbx_poly_seq_scheme.hetero
A 1 1 VAL 1 378 378 VAL VAL A . n
A 1 2 MET 2 379 379 MET MET A . n
A 1 3 ASN 3 380 380 ASN ASN A . n

```

Figure 3.2: Example of `_pdbx_poly_seq_scheme` showing all of 12 attribute headers and just the first 3 rows of the values for those attributes.

Parsing & tokenising mmCIF

- **Example snippet of `_atom_site`:**

This category has 20 attribute names, listed vertically from the top down. Their 20 corresponding attribute values are below the names arranged horizontally like columns in tabular format.

```

loop_
_atom_site.group_PDB
_atom_site.id
_atom_site.type_symbol
_atom_site.label_atom_id
_atom_site.label_alt_id
_atom_site.label_comp_id
_atom_site.label_asym_id
_atom_site.label_entity_id
_atom_site.label_seq_id
_atom_site.pdbx_PDB_ins_code
_atom_site.Cartn_x
_atom_site.Cartn_y
_atom_site.Cartn_z
_atom_site.occupancy
_atom_site.B_iso_or_equiv
_atom_site.pdbx_formal_charge
_atom_site.auth_seq_id
_atom_site.auth_comp_id
_atom_site.auth_asym_id
_atom_site.auth_atom_id
_atom_site.pdbx_PDB_model_num
ATOM  1  N  N  .  VAL  A  1  1  ?  15.679  37.289  73.148  1.00  40.31  ?  378  VAL  A  N  1
ATOM  2  C  CA .  VAL  A  1  1  ?  15.954  36.969  71.752  1.00  38.04  ?  378  VAL  A  CA  1
ATOM  3  C  C  .  VAL  A  1  1  ?  15.410  38.133  70.933  1.00  40.69  ?  378  VAL  A  C   1

```

Figure 3.3: Example of `_atom_site` showing all of 20 attribute headers and just the first 3 rows of the values for those attributes.

Parsing & tokenising mmCIF

- **Parsing mmCIF:**

- The two aforementioned categories are read into **Python** dicts.
- The dicts are converted to **Pandas** dataframes with one dataframe per protein.
Pandas dataframes provide a simple and reliable **SQL**-based API for manipulating, further parsing and manually inspecting the data, that is now in a tabular format with column names.
- The two categories (in 2 dataframes) are joined, all categories except for 8 are removed.
- Non-protein rows are removed. E.g. if `_atom_site.group_PDB` is `HETATM` instead of `ATOM`, remove entire row.
- Each column is cast to appropriate data type (such as numeric or string).
- Low occupancy rows are removed, typically where ≤ 0.5 .
Occupancy represents the fraction of time or probability that a given atom is present at the given coordinates within the crystal structure (or vitrified state in cryo-electron microscopy).
- Rows with missing structural coordinates are removed.

S_asym_id	S_seq_id	S_mon_id	A_id	A_label_atom_id	A_Cartn_x	A_Cartn_y	A_Cartn_z
A	5	SER	1	N	11.518	-2.609	10.209
A	5	SER	2	CA	10.437	-2.988	11.166
A	5	SER	3	C	9.932	-4.412	10.885
A	5	SER	4	O	10.388	-5.08	9.951
A	5	SER	5	CB	9.289	-1.972	11.101
A	5	SER	6	OG	8.65	-1.976	9.828
A	6	ALA	7	N	9.006	-4.884	11.713
A	6	ALA	8	CA	8.417	-6.208	11.526
A	6	ALA	9	C	7.574	-6.236	10.25
A	6	ALA	10	O	7.509	-7.25	9.555
A	6	ALA	11	CB	7.574	-6.583	12.739
A	7	GLU	12	N	6.937	-5.106	9.947
A	7	GLU	13	CA	6.138	-4.955	8.737

Table 3.3: Example of the 8 attributes in 8 columns of a dataframe, as returned by `cif_parser.parse_cif()`, showing first 13 rows. Column names prefixed with ‘A.’ are from `_atom_site`. Column names prefixed with ‘S.’ prefix are from `_pdbx_poly_seq_scheme`.

`cif_parser.parse_cif()` returns a **Python** list of **Pandas** dataframes, one for each PDBid and a single chain (PDBid_chain)

Parsing & tokenising mmCIF

- **Parsing mmCIF:**

Attributes	Description
S_asym_id	Protein chain
S_seq_id	Residue position
S_mon_id	Residue name (3-letter)
A_id	Atom position
A_label_atom_id	Atom name
A_Cartn_x	X coordinate
A_Cartn_y	Y coordinate
A_Cartn_z	Z coordinate

Table 3.4: Description of the 8 mmCIF attributes that are parsed and will be used for tokenisation in preparation for model training. Attribute names shown here are the same as in the original mmCIF but pre-fixed with ‘S’ or ‘A’ to signify which of the two categories they come from, namely `_pdbx_poly_seq_scheme` or `_atom_site`, respectively. These prefixed names are the column names of the dataframes and `ssv` files.

- **Tokenising parsed mmCIF:**

Tokenising the 8 parsed categories of the mmCIFs was performed as follows:

- Keep only specified chain, otherwise keep all (for now).

If `PDBid_chain` is specified, e.g. ‘1AD6_A’, instead of `PDBid` e.g. ‘1AD6’, then any other chains in the mmCIF are discarded.

- All hydrogen atoms are removed.
- Any chains that are not protein are removed.
- Any chains that do not have enough backbone atoms are removed.
- All chains except for one is removed for each mmCIF.

(The selection criteria of which chain to keep is arbitrary in the current implementation.)

- Positions of one “anchor” backbone atom per residue are recorded.
- Atoms and residues are enumerated by mapping to an ordinal encoding.
- Results are written to `ssv` files.
- Missing values are profiled.

- **Where a specific chain is included, keep only that one, otherwise keep all chains:**

The format of PDBids expected

- **All hydrogen atoms are removed:**

A list of 45 hydrogen atom names is in `data/enumeration/hydrogens.lst`. (They are H, H2, HA, HA2, HA3, HB, HB1, HB2, HB3, HD1, HD2, HD3, HD11, HD12, HD13, HD21, HD22, HD23, HE, HE1, HE2, HE3, HE21, HE22, HG, HG1, HG2, HG3, HG11, HG12, HG13, HG21, HG22, HG23, HH, HH2, HH11, HH12, HH21, HH22, HXT, HZ, HZ1, HZ2, HZ3). The limited number of hydrogen atoms in the mmCIFs are removed according to this list.

Parsing & tokenising mmCIF

- **Any chains that are not protein are removed:**

The functionality is essentially already performed by `_make_new_bckbone_or_sdchain_col()` called prior to this step, because it assigns `pandas.NA` to any atoms that are neither backbone atoms (N, CA, C, O, OXT), nor side-chain atoms (CB, CD, CD1, CD2, CE, CE1, CE2, CE3, CG, CG1, CG2, CH2, CZ, CZ2, CZ3, ND1, ND2, NE, NE1, NE2, NH1, NH2, NZ, OD1, OD2, OE1, OE2, OG, OG1, OG2, OH, SD, SG) so if there are any `pandas.NA` values in the `bb_or_sc` dataframe column, then the whole chain is removed on the assumption that this chain is not a protein chain.

- **Any chains that do not have enough backbone atoms are removed:**

Currently the limit is 0, such that if any residues at all of a protein chain have no backbone atom coordinates in the mmCIF, then the whole chain is removed from the dataset.

- **Positions of one “anchor” backbone atom per residue are recorded:**

The backbone atoms of a residue include N, CA, C, O, OXT. Tokenisation required that the index of one “anchor” backbone atom per residue be stored. The alpha-carbon (CA) was chosen to be this anchor atom. It was observed, albeit rarely, that a residue might lack the coordinates for its alpha-carbon. In this circumstance I assign the most C-terminal of the other available backbone atoms of the given residue.

- **Atoms and residues are enumerated by mapping to an ordinal encoding:**

The 20 amino acids are mapped to integers (0 to 19) in `data/enumeration/residues.json`, using the three-letter amino acid format rather than FASTA 1-letter format (to reduce risk of undetected errors due to typos). According to <https://www.ebi.ac.uk/pdbe-srv/pdbechem/atom/list/<from ARG to TRP>>, the 20 amino acids are composed of 83 atoms. These were manually read and saved to `data/enumeration/unique_atoms_only.json`. Hydrogen atoms were removed from this list, leaving 38 unique atoms, mapped to integers (0 to 37), stored in `data/enumeration/unique_atoms_only_no_hydrogens.json`.

- **Results are written to ssv files:**

The end product of tokenisation by `tokeniser.parse_tokenise_write_cifs_to_flatfile()` is written to a flat file (e.g. ssv) in `src/diffusion/diff_data/tokenised`. Each protein’s tokenised data, which is in the form of a Pandas dataframe, is written to a flat file, currently an ssv. For each run, the function must also write one new `lst` file to `src/diffusion/diff_data/PDBid_list`. This file contains a list of all the `PDBid_chains` that were successfully tokenised and saved to flat file and its name incorporates the number of proteins that remained in the dataset after tokenisation, e.g. `pdbchains_<number of proteins>.lst`.

S_asym_id	S_seq_id	S_mon_id	A_id	A_label_atom_id	A_Cartn_x	A_Cartn_y	A_Cartn_z	bb_or_sc	bb_atom_pos	aa_label_num	atom_label_num
A	1	VAL	1	N	15.679	37.289	73.148	bb	2	17	17
A	1	VAL	2	CA	15.954	36.969	71.752	bb	2	17	1
A	1	VAL	3	C	15.41	38.133	70.933	bb	2	17	0
A	1	VAL	4	O	14.925	39.102	71.522	bb	2	17	26
A	1	VAL	5	CB	15.249	35.651	71.37	sc	2	17	2
A	1	VAL	6	CG1	15.131	34.766	72.59	sc	2	17	11
A	1	VAL	7	CG2	13.878	35.922	70.795	sc	2	17	12
A	2	MET	8	N	15.496	38.07	69.606	bb	9	10	17
A	2	MET	9	CA	14.948	39.147	68.774	bb	9	10	1
A	2	MET	10	C	15.802	40.435	68.73	bb	9	10	0
A	2	MET	11	O	15.727	41.283	69.615	bb	9	10	26
A	2	MET	12	CB	13.526	39.452	69.223	sc	9	10	2

Table 3.5: Example output of `parse_tokenise_write_cifs_to_flatfile()`, written to ssv, showing first 13 rows.

Parsing & tokenising mmCIF

- **Missing values are profiled:**

Null values can and have lead to failed training with potentially obscure stack-traces. As such, this is an important potential bug to handle. `tokeniser.get_nums_of_missing_data()` performs an extensive search for the multiple possible formats of null/missing values. In addition to Python `None`, the null types searched for include: `pandas.isna`, `pandas.NA`, `pandas.NaT`, `numpy.nan`; the strings: `'`, `'na'`, `'NA'`, `'nan'`, `'NaN'`, `'NAN'`, `'none'`, `'None'`, `'null'` and `'Null'`. This helped identify unusable mmCIF, such that for a list of 573 single-domain proteins, for example, `tokeniser.py` removes 8 of them, generating a dataset of 565 tokenised PDBs for training with. (This approach, which could be referred to as 'defensive programming', is based on practical experience of PDB data, leading to the conclusion that one cannot assume that 100% of the data in 100% of files will follow a standard and consistent format. Errors of this type might be due to data entry or experimental data. (An additional function that is not yet implemented `tokeniser._each_column_has_expected_values()` would take this a stage further by checking that the numbers within each column fall within a valid range). A sanity-check of data coordinates is performed subsequently too in `load_dataset()` (lines 153-162 and 181-184).

Preparation of tokenised mmCIF for datasets

- **Loading 5 values for each PDBid_chain into 1 of 2 datasets:**

Model training is preceded by a final processing step of the tokenised mmCIF data, in order to create a Python list ('validation_list' or 'train_list') of a Python list ('sp') containing one Python tuple per mmCIF. This tuple contains 6 values: `aacodes`, `atomcodes`, `aaindices`, `bbindices`, `target`, and `target_coords`. (Note: the 2nd Python list, `sp`, has data from only one PDBid_chain in the current implementation, so `sp` is currently a one-element list, (making `train_list` a list of one-element lists). An alternative would be to modify the current implementation to expect the input file to contain multiple space-separated PDBid_chains per line, such that `sp` would instead contain values for all of those PDBid_chains.)

variable name	Description	Dimensions*
<code>aacodes</code>	Ordinal encoding of protein sequence	(150, 1)
<code>atomcodes</code>	Ordinal encoding of atoms of protein sequence	(1350, 1)
<code>aaindices</code>	zero-based index of protein sequence	(1350, 1)
<code>bbindices</code>	zero-based index of anchor backbone atoms	(150, 1)
<code>target</code>	PDBid_chain name	NA
<code>target_coords</code>	3-d Cartesian coordinates of atoms	(1350, 3)

Table 3.6: Descriptions of the 5 tokenised mmCIF values: `atomcodes`, `aaindices`, `bbindices` and `target_coords` and their expected *dimensions and values for one example PDBid_chain which has 150 residues and 1350 atoms.

- **Running the dataset loading script:**

To generate the datasets ('train_list' and 'validation_list'), the user should pass the path of a chosen PDBid_chain list file to the main function of the training script

`pytorch_protfold_allatomclustrain_singlegpu.main(targetfile_lst_path)` which calls `dataset_loader.load_dataset(targetfile_lst_path)`. However, this function can also be run directly for testing/demonstration via the `if __name__ == "__main__":` in `dataset_loader.py`.

`load_dataset()` checks that the length of each PDBid_chain, according to `aacodes`, is the same as the length of the protein language model embedding for that PDBid_chain, so the corresponding embeddings must be generated first, if not already present in `src/diffusion/diff_data/emb`. It can be commented out for testing/demonstration purposes of running `dataset_loader.load_dataset(targetfile_lst_path)`. in isolation, as the embeddings are not needed for the datasets.

Preparation of tokenised mmCIF for datasets

- **Converting tokenised data to 5 arrays for the datasets:**

The tokenised data in the `ssv` files contain redundant information which, intentionally, was not removed before writing to file. This was in order to facilitate easy sanity-checks by eyeballing the data, as well as simple programmatic checks.

For `aacodes` and `bbindices`, ‘de-duplication’ on the dataframe is performed. This is because each dataframe holds both residue-level and atom-level data, so residues are always duplicated over all the rows of its atoms. (Indeed, this reflects the way the data is recorded in `_atom_site` in the raw mmCIF). The required dimensions are easily generated via use of `Pandas drop_duplicates()` operation. For the other 3, the values can be taken from the dataframe as is, without de-duplication.

- `aa_label_num` to `aacodes`.

This is done via de-duplication on `S_seq_id`.

- `atom_label_num` to `atomcodes`.

- `S_seq_id` to `aaindices`.

`aaindices` is a zero-based index that should increment by 1 continuously for each residue, whereas `S_seq_id` are residue sequence positions starting from 1 and can often have gaps (due to lack of structural data). So the row-indices of the dataframe on `S_seq_id` ensures that it starts from 0 and increments by 1 for every change of sequence number.

- `A_id`, `bb_atom_pos` and `S_seq_id` are used to generate `bbindices`.

Atom positions `A_id` uses the same numbering as the anchor backbone position `bb_atom_pos`. The row-index where those two columns match provides the `bbindices`, which is then de-duplicated on `S_seq_id`.

- `A_Cartn_x`, `A_Cartn_y` and `A_Cartn_z` to `target_coords`.

`target_coords` are mean-corrected values. The correction of X coordinates uses the mean of X coordinates for all the atoms in that `PDBid_chain`; Likewise, Y coordinates are corrected by the mean of all the Y coordinates; Z coordinates by the mean of all the Z coordinates.

<code>aacodes</code>	<code>atomcodes</code>	<code>aaindices</code>	<code>bbindices</code>	<code>target_coords</code>
10	17	0	1	[-0.211, 11.2, -15.3]
17	1	0	9	[-0.68, 10.8, -14.0]
9	0	0	16	[-1.46, 12.0, -13.5]
9	26	0	24	[-1.47, 13.3, -14.1]
3	2	1	32	[0.501, 10.4, -13.2]
15	11	1	41	[0.079, 9.83, -11.8]
3	12	1	47	[1.45, 9.22, -10.8]
13	17	1	56	[2.22, 7.88, -11.7]

Table 3.7: Example of what the first 8 values of `aacodes`, `atomcodes`, `aaindices`, `bbindices` and `target_coordinates` might look like. (`target_coords` shown to 3 s.f.)

Protein language model embedding

A pre-trained protein language model (pLM) made available through the HuggingFace **transformers** package (<https://huggingface.co/ElnaggarLab/ankh-base>) was used for generating protein sequence embeddings that serve as part of the input for training the neural network model. **Ankh-base**, described as a general-purpose pLM, was created with a strong emphasis placed on data efficiency with “knowledge-guided optimisation”, which generates embeddings with fewer parameters (Elnaggar et al. 2020; Elnaggar et al. 2023). It is the most popular model out of the 7 Ankh models available at <https://huggingface.co/ElnaggarLab> with over 37K downloads.)

AutoModelForSeq2SeqLM are a family of sequence-to-sequence models built on top of the T5 pre-trained general purpose language models (Raffel et al. 2019), designed for downstream transfer learning tasks. In NLP, these models, with an encoder-decoder architecture, can be used for translation tasks, summarisation and autoregressive tasks. The Ankh models have been pre-trained on massive protein datasets in order to generate contextualised encodings of protein sequences likely to include evolutionary ‘knowledge’ applicable to protein structure prediction. They may well have learned representations of both more generalised, as well as nuanced, sequence-structure relationships. Here they are intended to be provide the diffusion model with a head start

The import and instantiation of the model is performed according to the accompanying HuggingFace ‘model card’, namely using **transformers.AutoModelForSeq2SeqLM**. A tokeniser configured for it must also be instantiated separately, before use. The term Auto refers to the model and tokeniser being automatically selected according to the name passed to it as a string, in this case “ElnaggarLab/ankh-base”.

The following snippets is from `src/pLM/plm_embedder.py`:

```
...
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
hf_tokeniser = AutoTokenizer.from_pretrained("ElnaggarLab/ankh-base")
hf_model = AutoModelForSeq2SeqLM.from_pretrained("ElnaggarLab/ankh-base")
hf_eval_model = hf_model.eval() # disables dropout. Keeps embeddings consistent
```

The tokenised inputs of an amino acid sequence are generated and passed as an input to the encoder, which is the first argument of the model. However, the model also expects an input to the decoder which is the second argument. As such, the same tensor is passed as the second argument as well, essentially as a placeholder because it does not get used at all here.

```
...
inputs = hf_tokeniser(amino_acid_sequence)
decoder_input_ids = inputs["input_ids"]
decoder_input_ids = decoder_input_ids[:, :-1] # removes 'End Of Sequence' token
lm_output = hf_eval_model(decoder_input_ids, decoder_input_ids)
embedding = lm_output.encoder_last_hidden_state
```

The embedding’s gradients are detached which means they will not get updated by backpropagation in the model training.

```
embedding = embedding.detach()
```

The embeddings are saved to file, and read in by `dataset_loader.load_dataset()`, as well as **DMPDataset** from `pytorch_protfold.allatomclustrain_singlegpu.py`, as inputs to the model for training.

Forward pass of neural network:

(Note: I've replaced constant variables with the actual values for brevity.)

The noise schedule for the diffusion model (defined in `DMPDataset`) provides a smooth non-linear transformation over 12 time steps, and applied to coordinates:

```
tsteps = torch.arange(0, 1, 1/12)
sig_max_r7 = (16 * 10)**(1/7)
sig_min_r7 = 4e-4**(1/7)
noise_levels = (sig_max_r7 + tsteps * (sig_min_r7 - sig_max_r7))**7
random_noise = torch.randn_like(batched_coors) # <----- Gaussian noise
noised_coors = noise_levels.view(12, 1, 1) * random_noise + batched_coors
```

The following is a streamlined copy of `nndef_protfold_atompvt2.py`, with important omissions, to provide a compact overview which represents a large and complicated number of operations occurring over the computation graph (deep neural network), while adhering to the original Python to aid relating back the original script). The atom-level encoding is combined with residue-level encoding, via an element-wise sum (on 6 lines highlighted by '<--'):

```
class DiffusionNet(nn.Module):
def forward(embed, aacodes, atomcodes, aaindices, noised_coors, noise_levels):
...
posbias = AlibiPositionalBias(16).forward() # positional bias for attention
embed = nn.LayerNorm(1024)(embed)
seq_enc8 = nn.ModuleList(list of 8 x [SeqEncoderLayer(1024, 16)]) # attention
embed = seq_enc8(embed, posbias) # 3 cycles of this
_embed = nn.LayerNorm(256)(nn.Linear(1024, 256)(embed))
atom_embed = _embed[:, aaindices] # <----- Expand sequence embedding to atoms
coordscale = (noise_levels.view(-1, 1, 1).pow(2) + 256).sqrt()
atom_embed = (atom_embed
               + nn.Embedding(20, 256)(aacodes)[aaindices] # 20 amino acids <--
               + nn.Embedding(38, 256)(atomcodes) # 38 atoms <--
               + PositionalEncoding(256)(aaindices) <--
               + FourierEncodingLayer(256)(noise_levels) <--
               + nn.Linear(3, 256)(noised_coors / coordscale)) <--
atom_embed = nn.LayerNorm(256)(atom_embed)
atom_embed = nn.Sequential(10x[AtomEncoderLayer(256,8)])(atom_embed) # attention
pred_denoised = (nn.Sequential(nn.LayerNorm(256), nn.Linear(256))(atom_embed)
                 * 16 * (noise_levels / (256 + noise_levels**2)).sqrt())
                 + noised_coors * 256 / (256 + noise_levels**2))
pred_coors = nn.Linear(1024, 3)(embed)
pred_confs = nn.Linear(1024, 1)(embed)
return pred_denoised, pred_coors, pred_confs
```

The main `nn.Module` `DiffusionNet` utilises the five other `nn.Module` classes mentioned above (namely `AlibiPositionalBias`, `SeqEncoderLayer`, `PositionalEncoding`, `FourierEncodingLayer`, and `AtomEncoderLayer`). `AtomEncoderLayer` and `SeqEncoderLayer` perform multi-head attention (MHA) and both use another `nn.Module` called `SwiGLU` which serves as an activation function for these two modules. `SeqEncoderLayer` utilises a separate `nn.Module` for its MHA computation, namely `MultiheadAttention`, while `AtomEncoderLayer` implements MHA directly.

Note: This script was used for training the model on protein sequence inputs, but it is almost identical to the original script written by Prof David Jones for RNA sequences.

- **Calculating the loss:**

Neural networks learn according to minimising an objective function that is a measure of the differences between the network's output (i.e. prediction) and ground-truth (i.e. 'target'). As such, how the loss is defined determines what the neural network is actually designed to learn. Here, the loss is calculated as a weighted sum of three different loss equations, in `pytorch_protfold_allatomclustrain_singlegpu.py`. The `forward()` operation of `DiffusionNet` returns the three values needed for the loss calculations. Specifically: `pred_denoised, pred_coords, pred_confs = DiffusionNet.forward(<inputs>)`.

Backbone loss is the element-wise mean squared error between Euclidean pairwise distance matrix of predicted coordinates and target coordinates, per iteration of training loop:

```
predmap = torch.cdist(pred_coords, pred_coords)
targmap = torch.cdist(bb_coords.unsqueeze(0), bb_coords.unsqueeze(0))
bb_loss = F.mse_loss(predmap, targmap)
```

Confidence loss is the difference between predicted confidence scores and the structural quality metric, LDDT (Mariani et al. 2011):

```
diffmap = (targmap - predmap).abs().squeeze(0)
incmask = (targmap < 15.0).squeeze(0).fill_diagonal_(0)
incmask_denom = torch.clip(incmask.sum(dim=0), min=1)
lddt = (0.25*((diffmap<0.5)+(diffmap<1.0)+(diffmap<2.0)+(diffmap<4.0))*
            incmask).sum(dim=0) / incmask_denom)
conf_loss = (torch.sigmoid(pred_confs).squeeze(0) - lddt).abs().mean()
```

Diffusion loss is the difference between predicted and target coordinates at an individual step of the 'reverse' Markov chain (recall equation (1.4) in Introduction).

This difference must be rotation invariant to be effective. As such, the target is rotated to the best superposed alignment with the denoised prediction (using their 3D coordinates).

Each call to the `forward()` method of the diffusion network is a single step in the Markov chain, as the model learns to reconstruct protein structure starting from a normal distribution (i.e. the 'reverse' direction).

(A noise weighting score (`nwts`) is included to make the calculation more robust to a broader range of possible noise levels.)

```
rot_targets = lsq_fit(target_coords, pred_denoised)
nwts = (noise_levels.pow(2)+16**2) / (noise_levels + 16).pow(2)
diff_loss=((rot_targets-pred_denoised).pow(2).sum(dim=2).mean(dim=1)*nwts).mean()
```

The superposition (performed in `lsq_fit()`) is done by minimising the squared distances via finding the optimal rotation matrix via singular value decomposition:

```
...
cov = torch.matmul(P, Q.transpose(1, 2))
U, S, Vh = torch.linalg.svd(cov)
...
```

Note: This loss function was used for training the model on protein sequence inputs, but it is almost identical to the original script written by Prof David Jones for RNA sequences.

Training loop and backpropagation:

As mentioned in the previous section, training a neural network is to some extent defined by the loss function used. The step-by-step calculation of the gradient of this loss function with respect to the trainable parameters provides for the step-by-step updates to these parameters as it navigates down the loss landscape. The speed and effectiveness of the route by which it navigates depends on the learning rate and the algorithm of the optimiser. The optimiser currently used here is a variant of the highly effective Adam optimiser (Kingma & Ba 2014) which includes a term to rectify the variance of the adaptive learning rate (Liu et al. 2019). Attention and diffusion models can be unstable in early stages of training so the use of this rectified Adam optimiser may be helping to counter this.

```
...
network = DiffusionNet(...)
optimizer = torch.optim.RAdam(network.parameters(), max_lr)
```

As the gradients of the loss function with respect to the parameters are calculated for the many layers of the deep network of `DiffusionNet` by the chain rule, the updates backpropagate from the atom-level parameters in the top layers of the network model, down to the residue-level parameters in the bottom layers of the model just after the inputs. This is performed by the `.backward()` and `.step(optimizer)` functions for each protein sequence and repeated 2000 times for the whole dataset.

```
for epoch in range(2000):
    train_err, train_samples = 0.0, 0
    network.train()
    for batch in train_data_loader:
        optimizer.zero_grad()
        batch_loss = 0
        for sample in batch:
            batch_loss += calculate_sample_loss(sample)
        scaling_factor = batch_loss / len(sample_batch)
        scaler.scale(scaling_factor).backward() # <----- backpropagation
        scaler.step(optimizer) # <----- update parameters
        scaler.update()
    ...
```

Note: This training loop and regime was used for training the model on protein sequence inputs, but it is almost identical to the original script written by Prof David Jones for RNA sequences.

Chapter 4

Experiments

Experiments amounted to software engineering of the tokenisation pipeline; modification of the dataset-dataloader inputs; and minor changes to the training model.

Detailed instructions on running the tokeniser, embedder and model training scripts are provided in the `README.md` at https://github.com/zibae-2021/MSc_2024_project which has been set to ‘public’ visibility. As is mentioned in the `README.md`, the shell scripts will only run from a terminal that does **NOT** already have the conda environment activated.

A strategy for storage and transfer of larger data files (e.g. model weights) has not been developed to allow running the programs from anywhere. So the code is currently only set up to run from `joe-desktop` machine in Darwin computer lab.

The tokeniser, embedder and model training scripts can either be all launched from a single shell script or individually as follows:

- **Launching via shell scripts**

`cd` to top level of the project directory (see Figure 3.1 for directory structure overview).

- `sh launchers/tokenise_embed_train.sh`

Alternatively, individually:

- `sh launchers/_1_run_tokeniser.sh`
- `sh launchers/_2_build_embeddings.sh`
- `sh launchers/_3_train_model.sh`

Consult the `README.md` for details regarding changes to the default settings such as use of pre-written data files versus generating new ones.

- **Proof-of-principle small dataset training**

Model training using the relatively small dataset of 565 single-domain proteins was performed to completion of the training regime, which is currently set at 2000 epochs. The training pipeline was run to completion on this dataset a total of 3 times.

Chapter 5

Results

Model training from 2000 epochs was performed on a dataset of 565 single domain proteins (as described in Methods) and repeated twice more. This served as a proof-of-principle result, demonstrating that the tokenisation for protein sequences was fully debugged and integrated into the adapted version of the pre-existing RNA structure prediction pipeline.

The shape of the training loss curve in Figure 5.1 indicates that the model is learning, while that of the validation loss curve indicates the model is not overfitting and therefore improving in terms of generalisation.

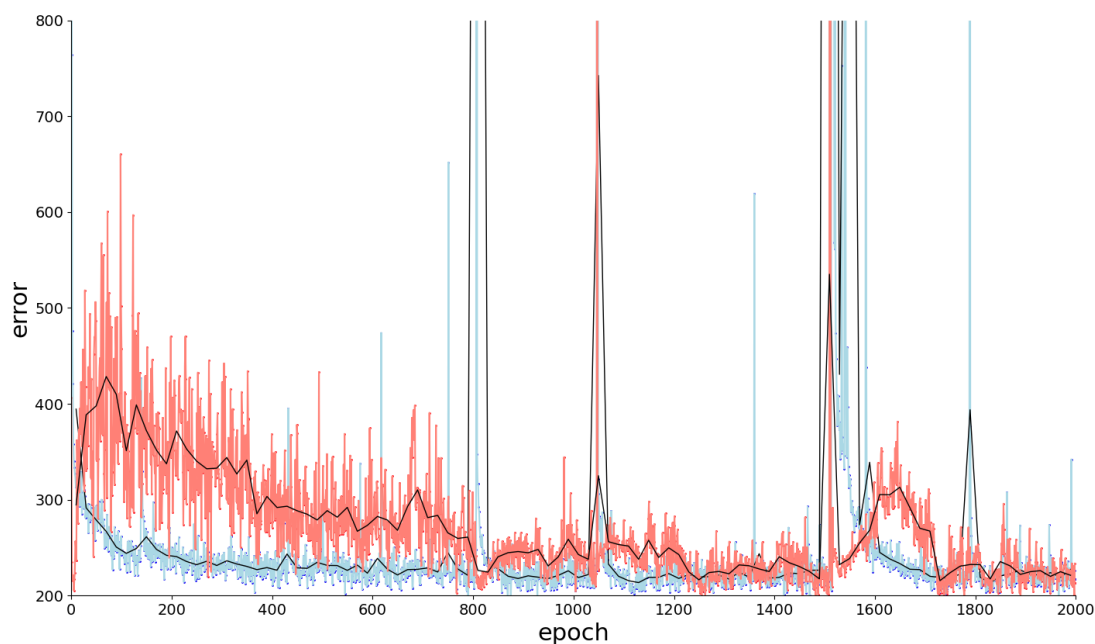


Figure 5.1: Train (blue) and validation (orange) loss curves over 2000 epochs.

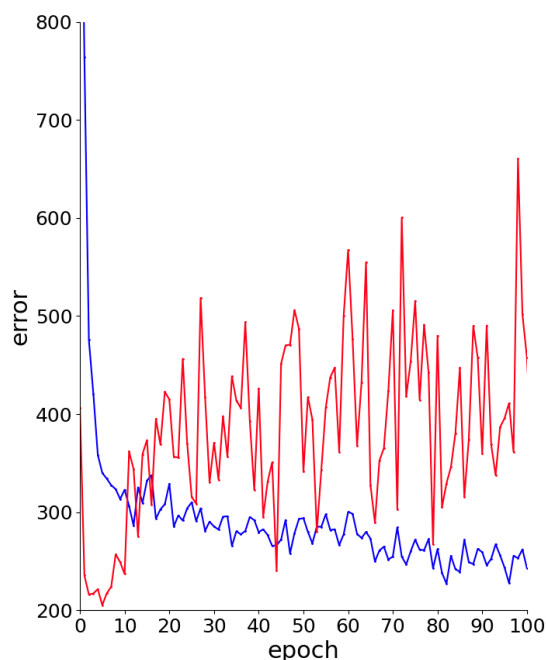


Figure 5.2: The first 100 epochs only from Figure 5.1.

Epoch	Train	Val
0	1109.4	406.25
1	764.4	235.38
2	475.9	216.00
3	420.9	217.00
4	358.4	221.38
5	340.0	205.12
6	334.1	217.12
7	327.5	223.88
8	323.4	257.00
9	313.3	248.88
10	322.8	236.88
11	306.2	361.75
12	286.2	344.00
13	325.3	275.25
14	309.4	359.25
15	332.2	373.50
16	337.5	307.50
17	293.3	395.00
18	302.8	369.00
19	308.1	422.75
20	328.8	415.00

Table 5.1: Train and validation losses of the first 21 epochs, highlighting substantial drops immediately after the first few epochs, which are not overtly obvious from the plots alone.

It is not yet clear why the losses spike late on in the training particularly at around 1500-1600 epochs. The losses are observed though to return to the levels they were before the spikes. Apart from this, these preliminary proof-of-principle results indicate that it would be suitable for the `Python` scripts (described in Methods) to be run on a scaled-up dataset, such as for at least 5,000 - 10,000 protein sequences.

Chapter 6

Discussion

A deep neural network designed and built in-house aims to predict 3D atomic structure of RNA molecules by a combination of pre-trained RNA-LLM embeddings and diffusion model, ‘glued’ together by multi-head attention. The primary goal of the work included in this project was to adapt this to proteins, train it on a small proof-of-principle dataset and use this for establishing what alterations can be made to simplify and optimise it further. Adapting it to protein sequences and incorporating Ankh pLM embeddings has been completed. Performing training on a small proof-of-principle dataset on a single GPU was also completed. Programs have been written that automate generating a larger dataset according to specific user-defined requirements. The next steps, given more time, would therefore have been to scale up the training and interrogate the model design for optimisations.

- **Comparison to diffusion model literature**

In the absence of additional neural network processing and residue covariance inputs, diffusion models alone (including their own ‘internal’ neural networks) have not been demonstrated to be capable of learning the latent variable distribution accurately enough to map an unseen protein sequence to its correct 3D atomic structure *ab initio*. It’s uncertain if the research field as a whole is strongly motivated to try to achieve this now anyway, given the success of AlphaFold 2 architecture. It is not surprising too that the primary focus of diffusion models for protein structure has been *de novo* protein design given that this aligns more favourably with the strengths of a probabilistic generative model.

- **Comparison to diffusion model literature - pLM embeddings**

Diffusion models were highly effective at mapping language model embeddings (built using transformers) to distributions of image data. (Saharia et al. 2022). They were found to be more computationally efficient and produced higher quality images than similar tasks performed using VAEs (Ramesh et al. 2021; Ramesh et al. 2022). This stood out by their use of large language models (LLMs) trained on text-only rather than text-image pairs. The best performing LLM used the T5 architecture, which is the same architecture that the pLM used in this project (Ankh) is built on.

Surprisingly, I have not found other examples of protein structure-related diffusion models that incorporate pre-trained pLM embeddings inputs. Here, the embedding input is combined with the amino acid sequence, its corresponding atoms and raw atomic coordinates, and further processed through multi-head attention before being used for generating the denoised predictions. Diffusion models are often described as being unconditional, or being conditioned, or subject to constraints that guide the denoising steps. By analogy with the image synthesis study, the pLM embeddings here ‘condition’ the diffusion model. This combines brute-force compute (pre-computed for transfer learning) with medium-scale computational demands of a diffusion model, used to refine the covariance information encoded by the pLM.

- **Comparison to diffusion model literature - equivariance and molecular heuristics**

As described in the Methods section, the loss function includes alignment of predicted and ground-truth structures enforce equivariance and also calculates loss based on backbone pairwise distances which is also invariant to rotation and translations. Data augmentation, written into `DMPDataset` class includes random rotations of the augmented data, which is also performed in at least two of the studies briefly reviewed in Related Work (Abramson et al. 2024; Chu et al. 2024). As such the model negates the requirement for computationally-demanding and complex SE(3)-equivariant GNNs for example. The protein representation to the model is therefore akin to a 3D point cloud rather sequentially-ordered and fixed chain of rotamers for example as in (Lin & AlQuraishi 2023; Chu et al. 2024; Abramson et al. 2024). None of the geometric constraints intrinsic to a polypeptide backbone have been injected into the protein sequence input as in (Lee, Kim & Kim 2023; Watson et al. 2023; Wu et al. 2024). And there is no attempt to consider the diffusion model as a form of molecular folding simulation as in (Wu et al. 2024).

- **Proposed follow-up experiments - scaling up, scaling out**

The dataset is used is very small by deep learning standards and by the number of parameters in the model being trained here. As already stated, it serves purely as a proof-of-principle. However, despite this, it still took 4 days to train 2000 epochs, on a single GPU. Prior to up-scaling this study, it would be important then to consider how to first accelerate training time as well as the bare minimum number of epochs in order to facilitate subsequent systematic experiments. Otherwise, a truly empirical exploration of an optimal training regime would become too time-consuming and computationally expensive. It is noted that experiments on dataset quality and size for training protein language models, the smaller but higher quality dataset outperformed the larger (Elnaggar et al. 2023). The model in this project already has a head-start by incorporating the pLM embeddings. It may be possible then that a ‘high quality’ dataset of less than 100K proteins, trained on an HPC architecture that distributes the training over more than one GPU would facilitate some systematic studies on optimisation of training efficiency.

- **Proposed follow-up experiments - systematic empirical interrogation of model**

In the case of AlphaFold2, extensive ablation studies were performed, revealing that individually each component of the full model had marginal contributions (Jumper et al. 2021). It was demonstrated that triangle attention and invariant point attention are not essential (Baek et al. 2023). Only the replacement of MSAs with single sequences resulted in a substantial decline in predictive performance. Further improvements in computational efficiency were identified subsequently, including the restricting the neural network processing of the MSA representation from 48 to 4 blocks (Abramson et al. 2024). These and other examples of the opportunities for increasing computational efficiency without sacrificing training efficacy support the proposed strategy of follow-up experiments in this vein, as also performed previously in (Elnaggar et al. 2023).

Thus, the follow-up experiments might include ablation studies, introduction of geometric constraints, dataset pruning, unfreezing pLM embeddings, and changes to the neural network trunk. In line with the last point mentioned, although transformers and newer applications of the attention mechanism are widely favoured in large-scale studies, it is not established that applying the convolution rather than attention mechanism is invariably suboptimal in every single context (Bai et al. 2021).

It is also proposed that such experiments would greatly benefit from the use of an ‘MLOps’ framework such as ‘Weights & Biases’. This would be preferable for systematic studies to facilitate an organised record of the statistics of training and inference results (Biewald 2020).

- **Further reading and denoising strategies**

Apart from the proposed experiments that are described above, there is also a great deal to be gained from an exhaustive review of the rapidly growing literature in the deep learning fields covered, particularly in the realms of recent studies with diffusion models. This would better inform heuristic and potentially novel modifications that could be as informative, if not more so, than blind ablation studies for example. In line with this, another possible avenue to improving the model might include modifications to the noising and denoising equations. Intriguing insights continue to emerge from this field and support further study on this for improving the efficiency of model training (Bansal et al. 2022; Daras et al. 2022).

- **Streamlining diffusion models and deep learning networks**

A crucial pre-requisite to all of the proposed follow-up work was to be able to reproducibly generate accurate inputs, devoid of ‘silent’ errors that might otherwise generate invalid results. That has been achieved. Inefficient models, let alone low quality datasets, have the potential to waste a great deal of computation (Strubell, Ganesh & McCallum 2019).

- **Closing remarks**

It has been proposed that a competition between heuristics and brute-force exists and has been lost to the latter on several occasions in recent history (Sutton 2019). This is obviously (and presumably deliberately) simplistic as it overlooks the nuances of how the success of learning on very deep networks, with massive datasets, has been facilitated by human-designed improvements to neural network design and curation of datasets. As such, the contributions of heuristics and brute-force have been interdependent but complementary. Unsupervised self-supervised brute-force will likely continue to drive major progress, but at the same time innovations may continue to reduce computational demand. Indeed this is highly topical as very recent events support the notion that necessity promotes innovation (DeepSeek-AI et al. 2024).

There is a strong incentive to promote less computationally-demanding deep learning models particularly for scientific research in academia. It is not unlikely that many of the epoch-defining challenges facing the modern age, from chronic illnesses to environmental sciences can be targetted by protein-based drugs and protein-based materials. As such, the importance of improving deep learning methods for protein structure prediction, and now design, cannot be understated.

Bibliography

- Anfinsen, Christian B. & Edgar Haber (1961). "Studies on the reduction and re-formation of protein disulfide bonds." In: *The Journal of biological chemistry* 236, pp. 1361–3.
- Ryle, AP et al. (1955). "The disulphide bonds of insulin." In: *Biochemical Journal* 60(4), p. 541.
- Kendrew, John C et al. (1958). "A three-dimensional model of the Myoglobin molecule obtained by X-ray analysis." In: *Nature* 181(4610), pp. 662–666.
- Perutz, Max F et al. (1960). "Structure of Hæmoglobin: a three-dimensional Fourier synthesis at 5.5-Å. resolution, obtained by X-ray analysis." In: *Nature* 185(4711), pp. 416–422.
- Moult, John et al. (1995). "A large-scale experiment to assess protein structure prediction methods." In: *Proteins: Structure* 23, pp. ii–iv.
- Bairoch, Amos & Rolf Apweiler (1997). "The SWISS-PROT protein sequence database: its relevance to human molecular medical research." In: *Journal of molecular medicine* 75(5), pp. 312–6.
- Bateman, Alex et al. (2024). "UniProt: the Universal Protein Knowledgebase in 2025." In: *Nucleic acids research*.
- "Crystallography: Protein Data Bank." (1971). In: *Nature New Biology* 233, p. 223.
- Berman, Helen M., Kim Henrick & Haruki Nakamura (2003). "Announcing the worldwide Protein Data Bank." In: *Nature Structural Biology* 10, pp. 980–980.
- Levinthal, Cyrus (1969). "How to fold graciously." In: *Mössbaun Spectroscopy in Biological Systems Proceedings*. Vol. 67(41). Univ. of Illinois Bulletin, pp. 22–24.
- Dill, Ken A. & Hue Sun Chan (1997). "From Levinthal to pathways to funnels." In: *Nature Structural Biology* 4, pp. 10–19.
- DF, Feng & Doolittle RF (1987). "Progressive sequence alignment as a prerequisite to correct phylogenetic trees." In: *J Mol Evol.* 25(4), pp. 351–60.
- Gotoh, Osamu (1999). "Multiple sequence alignment: algorithms and applications." In: *Advances in biophysics* 36, pp. 159–206.
- Katoh, Kazutaka et al. (2002). "MAFFT: a novel method for rapid multiple sequence alignment based on fast Fourier transform." In: *Nucleic acids research* 30(14), pp. 3059–66.
- Thomas, David J., Georg Casari & Chris Sander (1996). "The prediction of protein contacts from multiple sequence alignments." In: *Protein engineering* 9(11), pp. 941–8.
- Jones, David T. et al. (2012). "PSICOV: precise structural contact prediction using sparse inverse covariance estimation on large multiple sequence alignments." In: *Bioinformatics* 28(2), pp. 184–90.
- Blundell, Tom L., Bancinyane Lynn Sibanda & Laurence H. Pearl (1983). "Three-dimensional structure, specificity and catalytic mechanism of renin." In: *Nature* 304, pp. 273–275.
- Taylor, William R. (1986). "Identification of protein sequence homology by consensus template alignment." In: *Journal of molecular biology* 188(2), pp. 233–58.
- Blundell, Tom L. et al. (1988). "Knowledge-based protein modelling and design." In: *FEBS Journal* 172, pp. 513–520.

- Jones, David C., William R. Taylor & Janet M. Thornton (1992). “A new approach to protein fold recognition”. In: *Nature* 358, pp. 86–89.
- Qian, Ning & Terrence J. Sejnowski (1988). “Predicting the secondary structure of globular proteins using neural network models.” In: *Journal of molecular biology* 202(4), pp. 865–84.
- Bohr, Henrik G. et al. (1988). “Protein secondary structure and homology by neural networks The α -helices in rhodopsin.” In: *FEBS Letters* 241.
- Fredholm, Henrik et al. (1990). “A novel approach to prediction of the 3-dimensional structures of protein backbones by neural networks”. In: *FEBS Letters* 261.
- Fariselli, Piero & Rita Casadio (1999). “A neural network based predictor of residue contacts in proteins.” In: *Protein engineering* 12(1), pp. 15–21.
- Jones, David C. (1999). “Protein secondary structure prediction based on position-specific scoring matrices.” In: *Journal of molecular biology* 292(2), pp. 195–202.
- Fariselli, Piero et al. (2001). “Prediction of contact maps with neural networks and correlated mutations.” In: *Protein engineering* 14(11), pp. 835–43.
- Skwark, Marcin J. et al. (2014). “Improved Contact Predictions Using the Recognition of Protein Like Contact Patterns.” In: *PLoS Computational Biology* 10.
- Hochreiter, Sepp & Jürgen Schmidhuber (1997). “Long Short-Term Memory.” In: *Neural Computation* 9, pp. 1735–1780.
- Sutskever, Ilya, Oriol Vinyals & Quoc V. Le (2014). “Sequence to Sequence Learning with Neural Networks”. In: *ArXiv* abs/1409.3215.
- AlQuraishi, Mohammed (2018). “End-to-end differentiable learning of protein structure.” In: *bioRxiv*.
- Rumelhart, David E., Geoffrey E. Hinton & Ronald J. Williams (1986). “Learning representations by back-propagating errors.” In: *Nature* 323, pp. 533–536.
- Krizhevsky, Alex, Ilya Sutskever & Geoffrey E. Hinton (2012). “ImageNet classification with deep convolutional neural networks.” In: *Communications of the ACM* 60, pp. 84–90.
- He, Kaiming et al. (2015). “Deep Residual Learning for Image Recognition.” In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778.
- LeCun, Yann et al. (1989). “Backpropagation Applied to Handwritten Zip Code Recognition.” In: *Neural Computation* 1, pp. 541–551.
- Wang, Sheng et al. (2016). “Accurate De Novo Prediction of Protein Contact Map by Ultra-Deep Learning Model.” In: *PLoS Computational Biology* 13.
- Kandathil, Shaun M., Joe G. Greener & David T. Jones (2019). “Prediction of interresidue contacts with DeepMetaPSICOV in CASP13.” In: *Proteins* 87, pp. 1092–1099.
- Greener, Joe G., Shaun M. Kandathil & David T. Jones (2019). “Deep learning extends de novo protein modelling coverage of genomes using iteratively predicted structural constraints.” In: *Nature Communications* 10.
- Kryshtafovych, Andriy et al. (2019). “Critical assessment of methods of protein structure prediction (CASP)—Round XIII.” In: *Proteins: Structure* 87, pp. 1011–1020.
- Senior, Andrew W. et al. (2020). “AlphaFold: Improved protein structure prediction using potentials from deep learning.” In: *Nature* 577, 706 – 710.
- Bahdanau, Dzmitry, Kyunghyun Cho & Yoshua Bengio (2014). “Neural Machine Translation by Jointly Learning to Align and Translate”. In: *CoRR* abs/1409.0473.
- Vaswani, Ashish et al. (2017). “Attention is All you Need.” In: *Neural Information Processing Systems*.
- Baum, Leonard E. & Ted Petrie (1966). “Statistical Inference for Probabilistic Functions of Finite State Markov Chains.” In: *Annals of Mathematical Statistics* 37, pp. 1554–1563.
- Prince, Simon J.D. (2023). *Understanding Deep Learning*. The MIT Press.

- Ho, Jonathan, Ajay Jain & P. Abbeel (2020). “Denoising Diffusion Probabilistic Models.” In: *ArXiv* abs/2006.11239.
- Dhariwal, Prafulla & Alex Nichol (2021). “Diffusion Models Beat GANs on Image Synthesis.” In: *ArXiv* abs/2105.05233.
- Sohl-Dickstein, Jascha Narain et al. (2015). “Deep Unsupervised Learning using Nonequilibrium Thermodynamics.” In: *ArXiv* abs/1503.03585.
- Jarzynski, Christopher (1997). “Equilibrium free-energy differences from nonequilibrium measurements: A master-equation approach.” In: *Physical Review E* 56, pp. 5018–5035.
- Neal, Radford M. (1998). “Annealed importance sampling.” In: *Statistics and Computing* 11, pp. 125–139.
- Nichol, Alex & Prafulla Dhariwal (2021). “Improved Denoising Diffusion Probabilistic Models.” In: *ArXiv* abs/2102.09672.
- Song, Yang et al. (2020). “Score-Based Generative Modeling through Stochastic Differential Equations.” In: *ArXiv* abs/2011.13456.
- Song, Yang & Stefano Ermon (2019). “Generative Modeling by Estimating Gradients of the Data Distribution.” In: *Neural Information Processing Systems*.
- Guo, Zhiye et al. (2023). “Diffusion models in bioinformatics and computational biology.” In: *Nature reviews bioengineering* 2(2), pp. 136–154.
- Elnaggar, Ahmed et al. (2023). “Ankh : Optimized Protein Language Model Unlocks General-Purpose Modelling.” In: *bioRxiv*.
- Bengio, Yoshua, Aaron C. Courville & Pascal Vincent (2012). “Representation Learning: A Review and New Perspectives.” In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35, pp. 1798–1828.
- Bronstein, Michael M. et al. (2021). “Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges.” In: *ArXiv* abs/2104.13478.
- Kandathil, Shaun M. et al. (2020). “Deep learning-based prediction of protein structure using learned representations of multiple sequence alignments”. In: *bioRxiv*.
- (2022). “Ultrafast end-to-end protein structure prediction enables high-throughput exploration of uncharacterized proteins”. In: *Proceedings of the National Academy of Sciences of the United States of America* 119.
- Rives, Alexander et al. (2019). “Biological Structure and Function Emerge from Scaling Unsupervised Learning to 250 Million Protein Sequences.” In: *bioRxiv*.
- Elnaggar, Ahmed et al. (2020). “ProtTrans: Towards Cracking the Language of Life’s Code Through Self-Supervised Deep Learning and High Performance Computing.” In: *bioRxiv*.
- Bhattacharya, Nicholas et al. (2020). “Single Layers of Attention Suffice to Predict Protein Contacts.” In: *bioRxiv*.
- Kryshtafovych, Andriy et al. (2021). “Critical assessment of methods of protein structure prediction (CASP)—Round XIV.” In: *Proteins: Structure* 89, pp. 1607–1617.
- Jumper, John M. et al. (2021). “Highly accurate protein structure prediction with AlphaFold.” In: *Nature* 596, pp. 583–589.
- Figurnov, Michael (2021). *Lecture Slides, Slide 42*. Lecture presentation. URL: <https://www.youtube.com/watch?v=h0tH5AAUWGA>.
- Baek, Minkyung et al. (2021). “Accurate prediction of protein structures and interactions using a 3-track neural network.” In: *Science (New York, N.Y.)* 373, pp. 871–876.
- Robin, Xavier et al. (2021). “Continuous Automated Model EvaluatiOn (CAMEO)—Perspectives on the future of fully automated evaluation of structure prediction methods.” In: *Proteins* 89, pp. 1977–1986.

- Wu, Rui Min et al. (2022). “High-resolution de novo structure prediction from primary sequence.” In: *bioRxiv*.
- Li, Ziyao et al. (2022). “Uni-Fold Symmetry: Harnessing Symmetry in Folding Large Protein Complexes.” In: *bioRxiv*.
- Lin, Zeming et al. (2022). “Evolutionary-scale prediction of atomic level protein structure with a language model.” In: *bioRxiv*.
- Baek, Minkyung et al. (2023). “Efficient and accurate prediction of protein structure using RoseTTAFold2.” In: *bioRxiv*.
- Ahdritz, Gustaf et al. (2023). “OpenFold: Retraining AlphaFold2 yields new insights into its learning mechanisms and capacity for generalization.” In: *bioRxiv*.
- Hinton, Geoffrey E. (1986). “Learning distributed representations of concepts.” In: *Proceedings of the 8th Annual Meeting of the Cognitive Science Society*, pp. 1–12.
- Mikolov, Tomas et al. (2013). “Efficient Estimation of Word Representations in Vector Space.” In: *International Conference on Learning Representations*.
- Hjaltason, Gísli R. & Hanan Samet (2003). “Properties of Embedding Methods for Similarity Searching in Metric Spaces”. In: *IEEE Trans. Pattern Anal. Mach. Intell.* 25, pp. 530–549. URL: <https://api.semanticscholar.org/CorpusID:18292101>.
- Asgari, Ehsaneddin & Mohammad R. K. Mofrad (2015). “Continuous Distributed Representation of Biological Sequences for Deep Proteomics and Genomics.” In: *PLoS ONE* 10.
- Yang, Kevin K et al. (2018). “Learned protein embeddings for machine learning.” In: *Bioinformatics* 34, 2642–2648.
- Mazzaferro, Carlo (2017). “Predicting Protein Binding Affinity With Word Embeddings and Recurrent Neural Networks”. In: *bioRxiv*.
- Peters, Matthew E. et al. (2018). “Deep Contextualized Word Representations”. In: *ArXiv* abs/1802.05365. URL: <https://api.semanticscholar.org/CorpusID:3626819>.
- Heinzinger, Michael et al. (2019). “Modeling the language of life – Deep Learning Protein Sequences.” In: *bioRxiv*.
- Rao, Roshan et al. (2019). “Evaluating Protein Transfer Learning with TAPE.” In: *bioRxiv*.
- Harris, Zellig S. (1954). “Distributional Structure.” In: *WORD* 10(2-3), pp. 146–162.
- Yanofsky, C., Virginia Horn & Deanna Thorpe (1964). “Protein Structure Relationships Revealed by Mutational Analysis.” In: *Science* 146, pp. 1593–1594.
- Rao, Roshan M et al. (2020). “Transformer protein language models are unsupervised structure learners.” In: *bioRxiv*.
- Suzek, Baris E. et al. (2015). “UniRef clusters: a comprehensive and scalable alternative for improving sequence similarity searches.” In: *Bioinformatics* 31, pp. 926–932.
- Hayes, Thomas et al. (2024). “Simulating 500 million years of evolution with a language model.” In: *bioRxiv*.
- Devlin, Jacob et al. (2019). “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.” In: *North American Chapter of the Association for Computational Linguistics*.
- Raffel, Colin et al. (2019). “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer.” In: *J. Mach. Learn. Res.* 21, 140:1–140:67.
- Anand, N. & Tudor Achim (2022). “Protein Structure and Sequence Generation with Equivariant Denoising Diffusion Probabilistic Models.” In: *ArXiv* abs/2205.15019.
- Trippe, Brian L. et al. (2023). “Diffusion probabilistic modeling of protein backbones in 3D for the motif-scaffolding problem.” In: *ArXiv* abs/2206.04119.
- Watson, Joseph L et al. (2023). “De novo design of protein structure and function with RFdiffusion.” In: *Nature* 620, pp. 1089–1100.

- Lee, Jin Sub, Jisun Kim & Philip M. Kim (2023). “Score-based generative modeling for de novo protein design.” In: *Nature Computational Science* 3, pp. 382–392.
- Ingraham, John et al. (2023). “Illuminating protein space with a programmable generative model.” In: *Nature* 623, pp. 1070–1078.
- Wu, Kevin E. et al. (2024). “Protein structure generation via folding diffusion.” In: *Nature Communications* 15.
- Alamdari, Sarah et al. (2024). “Protein generation with evolutionary diffusion: sequence is all you need.” In: *bioRxiv*.
- Chu, Alexander E. et al. (2024). “An all-atom protein generative model.” In: *Proceedings of the National Academy of Sciences of the United States of America* 121.
- Abramson, Josh et al. (2024). “Accurate structure prediction of biomolecular interactions with AlphaFold 3.” In: *Nature* 630, pp. 493–500.
- Corso, Gabriele et al. (2022). “DiffDock: Diffusion Steps, Twists, and Turns for Molecular Docking.” In: *ArXiv* abs/2210.01776.
- Greener, Joe G., Lewis Moffat & David T. Jones (2018). “Design of metalloproteins and novel protein folds using variational autoencoders.” In: *Scientific Reports* 8.
- Anand, Namrata & Po-Ssu Huang (2018). “Generative modeling for protein structures.” In: *Neural Information Processing Systems*.
- Song, Yang & Stefano Ermon (2020). “Improved Techniques for Training Score-Based Generative Models”. In: *ArXiv* abs/2006.09011. URL: <https://api.semanticscholar.org/CorpusID:219708245>.
- Yang, Jianyi et al. (2019). “Improved protein structure prediction using predicted interresidue orientations”. In: *Proceedings of the National Academy of Sciences* 117, pp. 1496–1503.
- Lin, Yeqing & Mohammed AlQuraishi (2023). “Generating Novel, Designable, and Diverse Protein Structures by Equivariantly Diffusing Oriented Residue Clouds.” In: *International Conference on Machine Learning*.
- Dauparas, Justas et al. (2022). “Robust deep learning based protein sequence design using Protein-MPNN.” In: *Science (New York, N.Y.)* 378, pp. 49–56.
- Torres, Susana Vázquez et al. (2023). “De novo design of high-affinity binders of bioactive helical peptides.” In: *Nature* 626, pp. 435–442.
- Yim, Jason et al. (2023). “SE(3) diffusion model with application to protein backbone generation.” In: *ArXiv* abs/2302.02277.
- Orengo, Christine A. et al. (1997). “CATH—a hierarchic classification of protein domain structures.” In: *Structure* 5(8), pp. 1093–1108.
- Mariani, Valerio et al. (2011). “Assessment of template based protein structure predictions in CASP9.” In: *Proteins: Structure* 79.
- Kingma, Diederik P. & Jimmy Ba (2014). “Adam: A Method for Stochastic Optimization.” In: *CoRR* abs/1412.6980.
- Liu, Liyuan et al. (2019). “On the Variance of the Adaptive Learning Rate and Beyond.” In: *ArXiv* abs/1908.03265.
- Saharia, Chitwan et al. (2022). “Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding.” In: *ArXiv* abs/2205.11487.
- Ramesh, Aditya et al. (2021). “Zero-Shot Text-to-Image Generation.” In: *ArXiv* abs/2102.12092.
- Ramesh, Aditya et al. (2022). “Hierarchical Text-Conditional Image Generation with CLIP Latents.” In: *ArXiv* abs/2204.06125.
- Bai, Yutong et al. (2021). “Are Transformers More Robust Than CNNs?” In: *ArXiv* abs/2111.05464.
- Biewald, Lukas (2020). *Experiment Tracking with Weights and Biases*. Software available from wandb.com. URL: <https://www.wandb.com/>.

- Bansal, Arpit et al. (2022). “Cold Diffusion: Inverting Arbitrary Image Transforms Without Noise.” In: *ArXiv* abs/2208.09392.
- Daras, Giannis et al. (2022). “Soft Diffusion: Score Matching for General Corruptions.” In: *ArXiv* abs/2209.05442.
- Strubell, Emma, Ananya Ganesh & Andrew McCallum (2019). “Energy and Policy Considerations for Deep Learning in NLP.” In: *ArXiv* abs/1906.02243.
- Sutton, Richard (2019). *The Bitter Lesson*. URL: <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>.
- DeepSeek-AI et al. (2024). “DeepSeek-V3 Technical Report.” In: *ArXiv* abs/2412.19437.